

TECHNISCHE UNIVERSITÄT BERLIN

MASTER'S THESIS

Integrating Equality Saturation in the MimIR Compiler Framework

Author: Jannik Novak
Student ID: 392210

First Examiner:
Prof. Dr. Michel Steuwer
Second Examiner:
Prof. Dr. Roland Leißa

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Science in*
Computer Science

Compilers and Programming Languages
Institute of Software Engineering and Theoretical Computer Science

Berlin, September 25, 2026

Eigenständigkeitserklärung

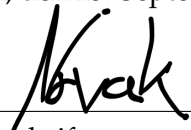
Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln wörtlich oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.

Sofern KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die Art der Nutzung (z. B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 8. März 2017 (https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMB/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf) habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, den 25. September 2026



Unterschrift

Erklärung zur Nutzung von KI-Tools

Bei der Anfertigung dieser Arbeit wurde ChatGPT (OpenAI) als unterstützendes Werkzeug verwendet. Die Nutzung umfasste insbesondere die sprachliche und stilistische Überprüfung von verfassten Texten sowie Vorschläge für alternative Formulierungen, Verbesserungen der sprachlichen Klarheit und die Identifikation mehrdeutiger oder unpräziser Formulierungen. Darüber hinaus wurde ChatGPT vereinzelt verwendet, um technische Aussagen und Erklärungen zu überprüfen.

Die Implementierung des in dieser Arbeit vorgestellten Systems, die Durchführung der Evaluation, die Interpretation der Ergebnisse sowie die daraus gezogenen Schlussfolgerungen wurden eigenständig erarbeitet. Die von ChatGPT erzeugten Vorschläge wurden vor einer möglichen Übernahme geprüft und gegebenenfalls angepasst oder verworfen.

Produkt: ChatGPT

Hersteller: OpenAI

Modell/Version: GPT-5.6 Sol

Art der Nutzung: Sprachliche und stilistische Überprüfung; Vorschläge für alternative Formulierungen; Überprüfung technischer Aussagen und Erklärungen.

TECHNISCHE UNIVERSITÄT BERLIN

Abstract

Faculty IV - Electrical Engineering and Computer Science
Institute of Software Engineering and Theoretical Computer Science

Master of Science

Integrating Equality Saturation in the MimIR Compiler Framework

by Jannik Novak

With the increasing importance of software optimizations for achieving higher computational performance, **DSLs** are utilized to generate highly optimized code for specialized hardware or application domains. A compiler optimization technique that has been applied to rewrite-based optimization of **DSLs** is equality saturation. Equality saturation facilitates the exploration of large transformation spaces through its use of e-graphs, which allow for the simultaneous and shared representation of many equivalent terms. MimIR is a compiler framework specifically designed to enable modeling a wide variety of **DSLs**, particularly those rooted in the lambda-calculus. Equality saturation is therefore relevant for the functional and expression-based languages modeled in MimIR. However, because MimIR did not previously provide a language-agnostic equality saturation mechanism, equality saturation had to be integrated separately for individual languages.

In this work, we implement an equality saturation plugin in MimIR that provides a language-agnostic mechanism for applying equality saturation to programs represented in MimIR. It extends MimIR with a set of **IR** constructs that can be used to configure and orchestrate an equality saturation backend. We implement two such backends based on the egg and slotted-e-graphs equality saturation libraries and integrate them into the plugin.

To achieve this, we devise a symbolic expression representation for MimIR and extend the MimIR compiler with a component for emitting programs in this representation. We implement two mechanisms for the construction of rewrite rules: rulesets, which consist of groups of rewrite rules defined directly in an equality saturation backend, and MimIR-defined rules, which are translated into corresponding backend rules at runtime. We then define corresponding representations of the MimIR language in egg and slotted-e-graphs and use the abstractions provided by the libraries to optimize input MimIR programs through equality saturation. Finally, the optimized MimIR program representation returned by a backend is reconstructed in MimIR through a dedicated phase.

To evaluate our equality saturation plugin, we reproduce map-fission and loop-tiling optimizations of RISE programs derived from existing literature. We find that the relevant subset of RISE primitives and rewrite rules can be modeled in MimIR and that every specified optimization target is reached, while the overhead introduced by the additional abstraction layer between MimIR and an equality saturation backend remains comparatively small.

Zusammenfassung

Mit der zunehmenden Bedeutung von Softwareoptimierungen für die Steigerung der Rechenleistung werden domänenspezifische Sprachen (DSLs) eingesetzt, um hochoptimierten Code für spezialisierte Hardware oder bestimmte Anwendungsdomänen zu generieren. Eine Compiler-Optimierungstechnik, die zur umschreib-basierten Optimierung von DSLs eingesetzt wird, ist Equality Saturation. Equality Saturation ermöglicht durch die Verwendung von E-Graphen die Exploration großer Transformationsräume, indem viele äquivalente Terme gleichzeitig in einer gemeinsamen Datenstruktur repräsentiert werden können. MimIR ist ein Compiler-Framework, das speziell für die Modellierung einer Vielzahl von DSLs entwickelt wurde, insbesondere solcher, die auf dem Lambda-Kalkül basieren. Dadurch ist Equality Saturation relevant für die in MimIR modellierten funktionalen und ausdrucksbasierten Sprachen. Da MimIR bisher jedoch keinen sprachunabhängigen Mechanismus für Equality Saturation bereitstellte, musste Equality Saturation für einzelne Sprachen separat integriert werden.

In dieser Arbeit implementieren wir ein Equality-Saturation-Plugin in MimIR, das einen sprachunabhängigen Mechanismus für die Anwendung von Equality Saturation auf in MimIR repräsentierte Programme bereitstellt. Es erweitert MimIR um eine Reihe von IR-Konstrukten, die zur Konfiguration und Orchestrierung eines Equality-Saturation-Backends verwendet werden können. Wir implementieren zwei solcher Backends auf Grundlage der Equality-Saturation-Bibliotheken *egg* und *slotted-egraphs* und integrieren diese in das Plugin.

Um dies zu erreichen, erstellen wir eine Repräsentation symbolischer Ausdrücke für MimIR und erweitern den MimIR-Compiler um eine Komponente zur Ausgabe von Programmen in dieser Repräsentation. Wir implementieren zwei Mechanismen zur Konstruktion von Rewrite Rules: Rulesets, die aus Gruppen von Rewrite Rules bestehen, welche direkt in einem Equality-Saturation-Backend definiert werden, und MimIR-definierte Rewrite Rules, die zur Laufzeit in entsprechende Backend-Regeln übersetzt werden. Wir definieren anschließend entsprechende Repräsentationen der MimIR-Sprache in *egg* und *slotted-egraphs* und verwenden die von den Bibliotheken bereitgestellten Abstraktionen, um MimIR-Eingabeprogramme durch Equality Saturation zu optimieren. Schließlich wird die von einem Backend zurückgegebene Repräsentation des optimierten MimIR-Programms durch eine dedizierte Phase in MimIR rekonstruiert.

Um unser Equality-Saturation-Plugin zu evaluieren, reproduzieren wir aus der bestehenden Literatur abgeleitete Map-Fission- und Loop-Tiling-Optimierungen von RISE-Programmen. Wir stellen fest, dass die für die Beispiele relevanten RISE-Konstrukte und Rewrite Rules in MimIR modelliert werden können und dass alle spezifizierten Optimierungsziele erreicht werden, während der durch die zusätzliche Abstraktionsschicht zwischen MimIR und einem Equality-Saturation-Backend verursachte Overhead vergleichsweise gering bleibt.

Acknowledgements

I would like to thank my advisor, Michel, for coming up with the idea for this project, for always taking the time to discuss any questions that came up along the way, and for providing many helpful suggestions throughout the course of this work. I would also like to thank my second advisor, Roland, for always taking the time to answer my questions about MimIR, for reviewing my code contributions, and for helping me develop a better overall understanding of MimIR and where my work fits into it. Finally, I would like to thank my parents for their continued support throughout my studies.

Contents

Eigenständigkeitserklärung	iii
Erklärung zur Nutzung von KI-Tools	v
Abstract	vii
Acknowledgements	ix
1 Introduction	1
2 Background	5
2.1 MimIR	5
2.1.1 Calculus of Constructions	5
2.1.2 Core IR	6
2.1.3 IR construction	8
Mim	8
MimIR API	9
2.1.4 Plugins	10
Annexes	10
Normalization	11
2.1.5 Phases	12
2.2 Equality Saturation	13
2.2.1 Symbolic Expressions	14
2.2.2 Rewrite Rules	15
2.2.3 E-Graphs	16
E-Matching	17
Congruence Closure	17
2.2.4 Egg	18
Language Definition	18
Rewrite Rules	20
E-Graph Saturation	20
Term Extraction	21
E-Class Analysis	22
Conditional Rewrite Rules	23
2.2.5 Slotted E-Graphs	24
Limitations of Named Variable Representations	24
Slotted E-Graph Representation	26
3 Implementation	29
3.1 Overview	29
3.2 MimIR as Symbolic Expressions	30
3.2.1 Basic Symbolic Expression Representation	30
3.2.2 Encoding Lambda Nesting	32

3.2.3	Encoding Type Information	34
3.3	Equality Saturation Plugin	35
3.3.1	Plugin Interface	36
3.3.2	Configuration	37
	Mim	37
	MimIR API	38
3.3.3	Compile Pipelines	38
3.4	Equality Saturation Backends	39
3.4.1	MimIR Language Definition	40
3.4.2	Ruleset Embedding	42
3.4.3	MimIR Rule Translation	43
3.4.4	Equality Saturation Implementation	45
	E-Graph Construction	46
	E-Graph Saturation	47
	Term Extraction	48
3.4.5	Reachability Checks	48
3.4.6	Type Analysis	50
	Analysis Interface	50
	Type Inference	51
	Type Unification	53
	Limitations	54
3.4.7	Foreign Function Interface	55
3.5	Reconstructing MimIR	57
3.5.1	Rewrite Phase	57
	Interface	57
	Execution	58
	Configuration Parsing	59
3.5.2	Initialization	59
3.5.3	Conversion	62
3.5.4	Type Reconstruction	64
3.5.5	Limitations	65
4	Case Study: RISE	67
4.1	Overview	67
4.2	Encoding RISE in MimIR	68
4.3	Experimental Setup	69
4.4	Map Fusion and Fission	69
	Optimization Target	69
	Rewrite Rules	70
	Configuration	71
	Results	71
4.5	Loop Tiling	72
	Optimization Target	72
	Rewrite Rules	73
	Configuration	74
	Results	75
4.6	Discussion	75
5	Conclusion	79
	Bibliography	83

List of Figures

2.3	Distinct representations of the α -equivalent terms $\lambda x.\lambda y.x\ y$ and $\lambda y.\lambda x.y\ x$ in an e-graph	25
2.4	The term $\lambda x.\lambda y.x\ y$ represented in a slotted e-graph	27
3.1	High-level workflow for optimizing a MimIR program through equality saturation	29
3.2	Nesting tree computed for the main function	33
3.3	DAG representation of the initialization example	60
3.4	DAG representation of the conversion example	63

List of Tables

2.1	Internal <code>RecExpr<Arith></code> representation of the term $a * (1 + 0)$	20
4.1	Result of the map-fission optimization	71
4.2	Results of the loop-tiling optimizations	75

List of Listings

2.1	Typed literals	6
2.2	Dependent tuples and packs	7
2.3	Extraction and insertion	7
2.4	Dependent function type	7
2.5	Control flow via CPS functions	7
2.6	Lambda syntax sugar	8
2.7	Typed axiom	8
2.8	IR construction in Mim	9
2.9	IR construction using MimIR's C++ API	9
2.10	Arithmetic operations on natural numbers in the core plugin	10
2.11	Normalization of <code>%core.nat.add</code> in <code>normalize_nat()</code>	11
2.12	Constant folding with a rewrite phase	12
2.13	Definition of an arithmetic language using egg's <code>define_language!</code> macro followed by a term parsed as <code>RecExpr<Arith></code>	19
2.14	Definition of the rewrite rules $x + 0 \rightarrow x$ and $x * 1 \rightarrow x$ in egg	20
2.15	E-graph saturation using egg's Runner	21
2.16	Term extraction using egg's Extractor with the <code>AstSize</code> cost function	21
2.17	<code>make</code> function of the ConstantFolding Analysis	22
2.18	<code>merge</code> function of the ConstantFolding Analysis	23
2.19	A conditional rewrite rule $x/x \rightarrow 1$ if $x \neq 0$ expressed with egg's <code>rewrite!</code> macro	24
3.1	Example function <code>foo</code> defined in Mim	30
3.2	Desugared representation of <code>foo</code>	30
3.3	S-expression representation of <code>foo</code> for the egg backend	31
3.4	S-expression representation of <code>foo</code> for the slotted-egraphs backend	32
3.5	Mim representation of the main function with nested continuations	32
3.6	S-expression representation of the main function preserving its nesting structure	34
3.7	Function <code>foo</code> defined in Mim	34
3.8	Unannotated s-expression representation of <code>foo</code>	35
3.9	Type-annotated s-expression representation of <code>foo</code>	35
3.10	Configuration types and axioms of the eqsat plugin	36
3.11	Additional axioms of the eqsat plugin	36
3.12	Rewrite phase axioms	37
3.13	Example eqsat configuration in Mim	37
3.14	Example eqsat configuration through the MimIR API	38
3.15	Example Mim program extended with a custom compile pipeline	39
3.16	Excerpt of the MimIR language definition in egg	40
3.17	Excerpt of the MimIR language definition in slotted-egraphs	41
3.18	A variadic tuple represented with a cons list	41
3.19	The slot <code>\$foo</code> in scope for two variable uses	42

3.20	Excerpt of the ruleset represented by %eqsat.core	42
3.21	Interface for ruleset selection and rule retrieval	43
3.22	The rewrite rule %core.nat.add (?x, 0) → ?x defined in Mim	44
3.23	S-expression representation of the rewrite rule foo	44
3.24	Excerpt of the translation of a MimIR-defined rewrite rule to egg	45
3.25	Public interface for equality saturation in egg	46
3.26	E-graph construction	47
3.27	E-graph saturation	47
3.28	Term extraction	48
3.29	Public interface for reachability checks in egg	49
3.30	Equality saturation for reachability checking in check_reaches()	49
3.31	Reachability check in reach_hook()	50
3.32	Type analysis interface	51
3.33	Example Mim program for type inference	51
3.34	Type inference dispatch in make()	52
3.35	Tuple type inference	52
3.36	Example Mim program for type unification	53
3.37	Type unification via merge()	53
3.38	Type unification in unify()	54
3.39	FFI definitions for configuration enums in Rust	55
3.40	Generated C++ definitions for configuration enums	55
3.41	FFI representation of recursive expressions in Rust	56
3.42	Rewrite phase interface	57
3.43	Equality saturation orchestration in start()	58
3.44	Configuration parsing in import_config()	59
3.45	Example Mim function for the initialization traversal	59
3.46	S-expression representation of the initialization example	60
3.47	Iteration over recursive expressions during initialization	61
3.48	Initialization traversal of a recursive expression	61
3.49	Lambda initialization in init_lam()	62
3.50	Iteration over recursive expressions during conversion	63
3.51	Convert traversal of a recursive expression	64
3.52	Application reconstruction in convert_app()	64
3.53	Type reconstruction in create_type()	65
4.1	RISE primitives modeled in Mim	68
4.2	Map-fission optimization in Mim	70
4.3	Map fusion rule in the %eqsat.rise ruleset	70
4.4	Configuration for map fusion and fission	71
4.5	One-dimensional loop-tiling optimization in Mim	72
4.6	Loop-tiling rewrite rules in Mim	74
4.7	Configuration for loop tiling	74

List of Abbreviations

API application programming interface. 8–10, 12, 20, 23, 24, 34, 37, 38, 42, 44, 46, 79

AST abstract syntax tree. 14, 16, 21, 37, 38

CFG control-flow graph. 5, 7

CoC Calculus of Constructions. 1, 2, 5, 6, 15, 81

CPS continuation-passing style. 7, 8

DAG directed acyclic graph. 16, 60

DSL domain-specific language. vii, viii, 1, 2, 5, 8, 79, 81

FFI foreign function interface. 29, 39, 45, 48, 55, 56

IR intermediate representation. vii, viii, 1–3, 5, 6, 8–10, 12, 29, 30, 33, 37, 70, 79, 80

Dedicated to my parents

Chapter 1

Introduction

With the end of Dennard scaling and the resulting slowdown in improvements to single-processor performance, specialized hardware and corresponding software optimizations have become increasingly important for achieving higher computational performance. Particularly in the realm of high-performance computing, exploiting the capabilities of specialized hardware often requires abstractions and transformations that incorporate knowledge about the target architecture. **Domain-specific languages (DSLs)** address this challenge by exposing domain-specific abstractions that enable their compilers to generate highly optimized code for particular hardware architectures or application domains. (Esmailzadeh et al., 2011; Steuwer et al., 2022)

DSL compilers are frequently built on top of compiler frameworks such as LLVM and MLIR. When targeting LLVM, however, lowering the high-level abstractions of a **DSL** generally requires additional **intermediate representation (IR)** layers to translate them into the comparatively low-level constructs of LLVM **IR**. This increases the implementation effort for developers, who have to define these representation layers and the corresponding lowering passes for individual **DSL** compilers. (Leißa et al., 2025) MLIR aims to address this challenge through its concept of dialects, which enable the definition of domain-specific abstractions that are reusable across **DSLs**. (Lattner et al., 2021) Although this provides reusable components for representing domain-specific abstractions at different levels, the representation of higher-order functional languages in MLIR can still involve significant implementation effort and complexity. (Lücke, Steuwer, and Smith, 2021)

MimIR is a compiler framework recently developed by Leißa et al. (2025) that aims to address these remaining challenges. MimIR is rooted in the **Calculus of Constructions (CoC)** and natively supports higher-order functions, while additionally providing mechanisms that facilitate program analyses and transformations for higher-order programs. Similar to MLIR, MimIR achieves extensibility through its plugin architecture, which enables the definition of domain-specific abstractions as well as analyses and transformations at the **IR** level.

Although compiler frameworks such as MimIR provide the mechanisms for expressing domain-specific abstractions and transformations at the **IR** level, the effective application of these transformations to generate optimized code poses a separate challenge. In conventional compilers, optimizations are achieved through the application of a sequence of dedicated transformations. A problem that arises with this approach, however, is the phase-ordering problem. It states that the order in which transformations are applied directly influences the quality of the generated code. (Barthou and Touati, 2006) This is because certain transformations may be enabled by prior transformations, while other transformations may be prevented as a result of preceding ones. A naive brute-force approach to this problem is to explore all possible transformation sequences. This approach, however, results in a rapidly

growing number of possible transformation sequences and therefore has only limited practical applicability. (Willsey et al., 2021)

To address the challenges associated with exploring large transformation spaces, the equality saturation optimization technique was proposed by Tate et al. (2011). Equality saturation utilizes e-graphs that facilitate the exploration of large transformation spaces through the simultaneous and shared representation of many equivalent terms. This shared representation mitigates the phase-ordering problem by preserving the equivalent terms produced through the application of different transformations without committing to a particular sequence. Equality saturation has been applied to the software-based optimization of **DSLs** that are conducive to equational rewriting. This includes functional languages based on the lambda-calculus, whose expression-based structure is well suited for rewrite-based optimization. (Schneider et al., 2025; Koehler, Trinder, and Steuwer, 2022) Thus, equality saturation is a particularly relevant mechanism for the **CoC**-based MimIR compiler framework, whose functional **IR** lends itself to equational rewriting.

As equality saturation involves several complex components such as the e-graph data structure, it is not trivial to integrate into existing applications. To facilitate its integration into a variety of applications, the egg library was developed by Willsey et al. (2021). Egg is a general-purpose, language-agnostic library that provides reusable components and algorithms that greatly reduce the development effort involved in integrating equality saturation into an application. The language-agnostic nature of egg makes it particularly suitable for integrating equality saturation into MimIR, in which a variety of languages can be modeled in a common **IR**.

Although MimIR is particularly well suited for modeling higher-order functional languages at the **IR** level and many such languages could benefit from integrating equality saturation into their optimization procedures, MimIR did not previously provide a generic equality saturation mechanism with which a language modeled in MimIR could be optimized. As a consequence, equality saturation had to be integrated separately for individual languages rather than reusing a language-agnostic mechanism already built into MimIR.

In this work, we implement the eqsat plugin, which extends MimIR with a set of **IR** constructs and transformations with which an equality saturation backend can be configured and orchestrated directly through MimIR. To achieve this, we integrate an s-expression emitter into the MimIR compiler that produces a representation of MimIR, which serves as an abstraction layer between MimIR and an equality saturation library. We provide corresponding equality saturation backends based on the egg and slotted-egraphs libraries that are integrated into the plugin and responsible for optimizing MimIR programs. To enable the definition of rewrite rules that guide the equality saturation procedure, we provide two distinct mechanisms through which they can be defined as part of an equality saturation backend or directly in MimIR. Lastly, we implement a phase that reconstructs the optimized representations returned by the equality saturation backends as MimIR terms.

We evaluate the eqsat plugin by reproducing two rewrite-based RISE program transformations derived from existing literature. (Schneider et al., 2025; Koehler, Trinder, and Steuwer, 2022) To this end, we model the relevant RISE primitives through corresponding **IR** extensions in MimIR and express the rewrite rules involved in these transformations using both mechanisms for rule construction. We find that our plugin reaches every optimization target defined in these examples, while the overhead introduced by the added abstraction layer between MimIR and the equality saturation backends remains comparatively small and does not become a performance bottleneck.

We begin in [chapter 2](#) by providing the background knowledge required for understanding the remainder of this work. In [chapter 3](#), we present the implementation of the eqsat plugin in detail, beginning with the s-expression representation of MimIR, followed by the [IR](#) extensions of the eqsat plugin, the implementation of the equality saturation backends, and, finally, the reconstruction of optimized terms in MimIR. In [chapter 4](#), we evaluate the eqsat plugin on the selected RISE program transformations. Finally, [chapter 5](#) summarizes this work and its limitations and provides directions for future work.

Chapter 2

Background

In this chapter, we present the background knowledge required for understanding the remainder of this work. We introduce the MimIR compiler framework and its core **IR** constructs, explain the process of building a MimIR program, and discuss its plugin architecture. We then provide an overview of equality saturation and introduce the concepts and mechanisms relevant to our implementation.

2.1 MimIR

MimIR is a compiler framework recently developed by Leißa et al. (2025) that is rooted in the **CoC** and natively supports higher-order functions. The **CoC** is a highly expressive type system that we briefly introduce in the following subsection. MimIR additionally implements the novel compiler **IR** λ_G , introduced by Leißa and Griebler (2026), which is based on free-variable nesting and enables a wide range of program analyses and transformations for higher-order programs without relying on **control-flow graphs (CFGs)**. While conventional **CFGs** facilitate many analyses and transformations based on the dominance relation, this relation is not well-defined for higher-order programs. (Leißa and Griebler, 2026) MimIR achieves extensibility through its plugin architecture, which enables the definition of domain-specific abstractions at the **IR** level as well as domain-specific transformations and analyses. Together with MimIR’s textual representation, this allows for axiomatic extensions that involve minimal development effort.

In this section, we briefly introduce the **CoC** and outline MimIR’s core **IR** constructs. We then demonstrate the construction of a MimIR program with a concrete example before presenting the MimIR plugin architecture and explaining how these plugins tie into MimIR’s optimization model.

2.1.1 Calculus of Constructions

MimIR’s core is based on the **CoC**, a higher-order typed lambda calculus first introduced by Coquand and Huet (1988). The **CoC** combines all forms of abstraction represented in the Lambda Cube first described by Barendregt (1991). These are ordinary term-to-term abstractions, term-to-type abstractions found in dependent types, type-to-term abstractions characteristic of polymorphism, and higher-order type-to-type abstractions. (Pierce, 2004) The main reasons why the MimIR developers chose the **CoC** as the foundation of their **IR** are its expressiveness, which allows MimIR to host type systems of a wide variety of **DSLs**, and the simplicity that comes with having expressions as the only syntactic category. (Leißa et al., 2025)

The identity function shown in (2.1) constitutes an example of a term-to-term abstraction. The polymorphic identity function shown in (2.2) exemplifies a type-to-term abstraction. Example (2.3) shows a term-to-type abstraction where $Arr\ n$

denotes the type of arrays of length n . The function shown in (2.4) exemplifies a type-to-type abstraction.

$$\lambda x : T. x : T \rightarrow T \quad (2.1)$$

$$\lambda T. \lambda x : T. x : \forall T. T \rightarrow T \quad (2.2)$$

$$\lambda n : \text{Nat}. \text{Arr } n : \text{Nat} \rightarrow * \quad (2.3)$$

$$\lambda T. T \rightarrow T : * \rightarrow * \quad (2.4)$$

In the **CoC**, types are classified by a sort to distinguish between different levels of types. The first sort is symbolized by $*$ and classifies proper types. This sort is itself classified by the sort symbolized by $\square$. (Pierce, 2004) MimIR chooses to classify its types with the same notation in its textual format, but internally represents them using `Type`. In MimIR, `Type0` corresponds to the sort of proper types $*$, while `Type1` corresponds to the sort of kinds $\square$. We make use of this representation to accurately model MimIR programs as s-expressions in [subsection 2.2.1](#).

2.1.2 Core IR

In this subsection, we introduce the **IR** constructs that are most relevant to the remainder of this work. There are three kinds of literals in MimIR with the types `Nat`, `Bool`, and `Idx`. The `Nat` and `Bool` types represent natural numbers and Booleans, respectively, while the dependent `Idx` type classifies integer values by additionally specifying the number of its inhabitants. For example, the type `Idx 2` is inhabited by the values `0_2` and `1_2`. The `Bool` type is an alias for the type `Idx 2` and its values `tt` and `ff` are aliases for `0_2` and `1_2`. [Listing 2.1](#) shows a group of typed literals in MimIR’s textual representation `Mim`.

```

1 tt: Bool;
2 ff: Bool;
3 1_2: Idx 2;
4 2_3: Idx 3;
5 1: Nat;
6 2: Nat;
```

LISTING 2.1: Typed literals

In addition, MimIR provides dependent tuples and packs with their corresponding types: `sigma` and `array`. A dependent tuple consists of a heterogeneous collection of terms that can depend on prior terms in the same tuple. A dependent pack is a sequence of elements whose length is specified by an expression of type `Nat`. In `Mim`, tuples are denoted by rounded brackets, whereas sigma types are denoted by square brackets. Packs specify their length and body inside of single angle brackets, while array types specify their length and element type in double angle brackets. Packs and arrays of length n can also introduce a variable i of type `Idx n` using the syntaxes $\langle i:n; e \rangle$ and $\langle\langle i:n; T \rangle\rangle$ where i can appear in e and T , respectively. This enables the construction of more complex expressions, which we demonstrate in an example below. [Listing 2.2](#) shows examples of dependent tuples and packs with their corresponding sigma and array types.

```

1 (T: *, x: T): [* , T];
2 <2; 3>:      «2; Nat»;
3 <i: 2; 3>:   «i: 2; Nat»;

```

LISTING 2.2: Dependent tuples and packs

To access and modify the elements contained in dependent tuples and packs, MimIR provides the `extract` and `insert` operations whose respective textual representations in Mim are `#` and `ins`. The index provided to these operations is an `Idx`-typed value whose cardinality matches the length of the indexed expression. Listing 2.3 shows the `extract` and `insert` operations applied to a tuple.

```

1 (ff, 2)#0_2:      Bool;
2 ins (ff, tt) 0_2 1: [Nat, Bool];

```

LISTING 2.3: Extraction and insertion

Functions inhabit dependent function types called `pi` types. Listing 2.4 shows a `pi` type commonly encountered in MimIR. The domain of this type is a sigma containing a natural number `n` and an array `Ts` of `n` types. Its codomain is an array of length `n` that introduces the variable `i` and describes a heterogeneous collection of values whose types correspond to those specified in `Ts`. The curly brackets indicate that the corresponding parameters are implicit and inferred from the application context.

```

1 {n: Nat, Ts: «n; *»} → «i: n; Ts#i»;

```

LISTING 2.4: Dependent function type

MimIR uses direct-style functions and **continuation-passing style (CPS)** functions. A continuation is a function that is non-returning, which MimIR expresses with a codomain of the uninhabited `bot` ($\perp$) type. The purpose of **CPS** functions is to equip MimIR programs with a control-flow structure in which continuations are analogous to the basic blocks of a **CFG**. (Leißa et al., 2025) Listing 2.5 demonstrates this with a concrete example. The continuation `foo` marks the main entry point, and the expression `(f, t)#cond` is analogous to a conditional branch on the Boolean parameter `cond` to the basic blocks corresponding to the continuations `f` and `t`. These continuations reconverge by applying the return continuation parameter of `foo` in their respective bodies.

```

1 con foo(cond: Bool, return: Cn Nat) =
2   (f, t)#cond ()
3   where
4     con f() = return 1;
5     con t() = return 2;
6   end;

```

LISTING 2.5: Control flow via CPS functions

Direct-style and **CPS** functions are internally modeled as lambdas, but Mim provides syntactic sugar for continuations and their types denoted by `con` and `Cn`, respectively. Continuations with a return parameter and their types are denoted by `fun` and `Fn`, respectively. Lambdas are additionally equipped with a partial evaluation filter denoted by an `@` prefix following their parameter groups. The partial

evaluation filter is a Boolean term that can depend on the lambdas’s parameters. It determines whether the lambda should be normalized in an application context. This means that the substituted body of the lambda is inlined at the call site. (Leißa et al., 2025) By default, the partial evaluation filter is set to `tt` for direct-style lambdas and `ff` for **CPS** lambdas. Listing 2.6 illustrates three equivalent representations of the same lambda in Mim.

```
1 fun (x: Nat): Nat           = return x;
2 con (x: Nat, return: Cn Nat) = return x;
3 lam (x: Nat, return: Nat → ⊥)@ff: ⊥ = return x;
```

LISTING 2.6: Lambda syntax sugar

To allow for the definition of new primitives, MimIR provides the axiom construct, which is a typed, opaque declaration whose semantics are provided by associated normalizing functions, as we explain in more detail in subsection 2.1.4. Axioms are the primary mechanism by which the abstractions and operations of a **DSL** can be represented in MimIR. Listing 2.7 shows the definition of a typed axiom in Mim. In Mim, axioms are prefixed with a `%`, and the axiom `%foo.bar` is associated with the function type `Nat → Nat`. It can be used in any context where a regular lambda of the same type could be used.

```
1 axm %foo.bar: Nat → Nat;
```

LISTING 2.7: Typed axiom

In the following subsection, we demonstrate the different mechanisms for building a complete program from the presented **IR** constructs.

2.1.3 IR construction

There are two approaches to building a MimIR program which we introduce in this subsection. Firstly, MimIR’s surface language Mim can be used to define programs and test cases with minimal implementation effort. Mim is also used in the MimIR plugin system to define groups of related operations and abstractions, as we explain in subsection 2.1.4. Secondly, MimIR provides a C++ **application programming interface (API)** that a DSL compiler can target for code generation. MimIR is equipped with a backend capable of emitting textual LLVM IR that can be further processed into an executable. (Leißa et al., 2025)

Mim

We introduce a minimal example program and demonstrate its construction first in Mim and then through the MimIR **API**. Listing 2.8 shows the example program defined in Mim.

```

1 plugin core;
2
3 lam inc(a: Nat): Nat =
4   %core.nat.add (a, 1);
5
6 fun extern main(x: Nat): Nat =
7   return (inc x);

```

LISTING 2.8: IR construction in Mim

In this example, we define a main function that takes an input parameter x of type Nat . We additionally define the direct-style lambda inc , which increments its natural number input parameter a . The `extern` attribute of the main function marks it as a root of MimIR’s internal program graph. The increment is achieved with the `%core.nat.add` axiom, which is loaded with the `core` plugin in the first line. For this example, it suffices to know that plugins define collections of related operations, and the `%core.nat.add` axiom expresses an addition of natural numbers. In the main function, the previously defined `inc` lambda is applied to x , and the result of this application is inlined because the partial-evaluation filter of direct-style lambdas defaults to `tt`. Lastly, the main function applies its return continuation to return the incremented value.

MimIR API

Listing 2.9 demonstrates how the same program is constructed with the MimIR API.

```

1 auto driver = Driver("example");
2 auto& w     = driver.world();
3 ast::load_plugins(w, View<std::string>{"core"});
4
5 auto nat_t = w.type_nat();
6 auto inc   = w.mut_lam(nat_t, nat_t)->set("inc");
7 auto a     = inc->var();
8 auto args  = w.tuple(a, w.lit_nat(1));
9 auto add   = w.annex<core::nat::add>();
10 inc->set_body(w.app(add, args));
11
12 auto main  = w.mut_fun(nat_t, nat_t)->set("main");
13 auto x     = main->var(2, 0);
14 auto ret   = main->var(2, 1);
15 main->set_filter(false);
16 main->set_body(w.app(ret, w.app(inc, x)));
17 main->externalize();

```

LISTING 2.9: IR construction using MimIR’s C++ API

We begin by instantiating the `Driver`, which is responsible for loading plugins and tracking global compiler flags. Most importantly for this example, it contains the `World` in which the MimIR program graph is stored. The `World` is the primary interface for IR construction and provides factory methods for every IR node. We then create a reference `w` to the `World` and load the `core` plugin with the `ast::load_plugins()` function.

We obtain the `Nat` type using the `World`’s factory method `w.type_nat()` and store it in the variable `nat_t`. Then, we create a mutable lambda with the domain and codomain `Nat` via `w.mut_lam(nat_t, nat_t)` and assign it the name `"inc"`. MimIR

distinguishes between mutable and immutable expressions, where mutables may be recursively defined and introduce variables, while immutables are non-recursive and introduce no variables. Their construction differs in that mutables may be constructed incrementally with `set()` operations, while immutables must be constructed in a single step. We access the variable `a` of the `inc` lambda with `inc->var()` and construct the tuple `(a, 1)` via `w.tuple(a, w.lit_nat(1))`. Then, we load the `%core.nat.add` axiom via `w.annex<core::nat::add>()`. Finally, we set the body of this lambda as the application `%core.nat.add (a, 1)`.

We construct the `main` function similarly to `inc`, using the factory method `w.mut_fun(nat_t, nat_t)`. The type of the `main` function is structurally similar to the type of the desugared lambda we introduced in Listing 2.6. Its domain consists of a sigma type that contains the parameter `x` in its first position and a return continuation in its second position. We access these parameters with the respective calls `main->var(2, 0)` and `main->var(2, 1)`, which specify the variable's arity followed by the respective element index. Finally, we set the body of the `main` function as the curried application `return (inc x)` and mark it as `external`.

In this subsection, we demonstrated how Mim and the MimIR API can be used in different contexts to construct the same underlying IR. We also briefly introduced plugins and made use of the `%core.nat.add` axiom defined in the `core` plugin. In the following subsection, we describe the internal structure of plugins in greater detail and explain how they fit into the MimIR ecosystem.

2.1.4 Plugins

To facilitate the extension of MimIR's intentionally minimal native IR, MimIR features a plugin system that can be used to define a set of abstractions specific to a domain. The building blocks of a plugin are a Mim file containing the IR extensions exposed by the plugin and C++ code specifying the behavior of certain operations under normalization. These IR extensions are called annexes and we introduce them in more detail below. A plugin can also be equipped with a set of transformation phases. The purpose of these phases is to convert between different abstraction layers and to apply potentially domain-specific program optimizations. (Leißa et al., 2025) We explain the mechanisms of phases in greater detail in the following subsection.

Annexes

The axioms, functions, types, and variables defined as part of a plugin are called annexes. Annex names are prefixed with the name of their plugin. In the `core` plugin, for example, annex names begin with the `%core` prefix. In an earlier example, we added natural numbers with the axiom `%core.nat.add`. Listing 2.10 shows the definition of this axiom in the Mim file of the `core` plugin.

```
1 axm %core.nat(add, sub, mul, div, mod): «2; Nat» → Nat, normalize_nat;
```

mim

LISTING 2.10: Arithmetic operations on natural numbers in the `core` plugin

This axiom defines a common function signature for the arithmetic operations on natural numbers specified in parentheses: `add`, `sub`, `mul`, `div`, and `mod`. It inhabits a function type with a domain that consists of an array of 2 natural numbers and whose codomain is a single natural number. The domain is represented as an array

type rather than a sigma type because homogeneous tuples in MimIR are normalized upon construction to packs of the same length. Following the axiom type, the C++ function `normalize_nat()` is specified as a normalizer that encodes the semantics of these operations. In the following, we describe in more detail the purpose and behavior of normalization in MimIR.

Normalization

Normalization is used in MimIR to partially evaluate expressions immediately after their creation. A fundamental form of normalization in the lambda-calculus is β -reduction, which simplifies applications of lambda expressions through substitution of their variables. (Pierce et al., 2002) MimIR extends this notion of normalization to axioms, for which a normalizer specifies their behavior under application. Unlike lambdas, axioms are opaque because they lack an implementation and therefore may require a separate specification of their semantics. The normalization function `normalize_nat()`, which specifies the semantics of the natural number operations introduced in Listing 2.10, is defined in a C++ source file bundled with the core plugin. Listing 2.11 shows a condensed definition of this normalizer that highlights only the semantics of the `%core.nat.add` axiom.

```

1  template<nat id>
2  const Def* normalize_nat(const Def* type, const Def* callee, const Def* arg) {
3      auto& world = type->world();
4      auto [a, b] = arg->projs<2>();
5      if (is_commutative(id) && Def::greater(a, b)) std::swap(a, b);
6      auto la = Lit::isa(a);
7      auto lb = Lit::isa(b);
8
9      if (la) {
10         if (lb) {
11             switch (id) {
12                 case nat::add: return world.lit_nat(*la + *lb);
13                 // ...
14             }
15             // ...
16         return world.raw_app(type, callee, {a, b});
17     }

```

LISTING 2.11: Normalization of `%core.nat.add` in `normalize_nat()`

As soon as an application of `%core.nat.add` is constructed, the `normalize_nat()` function is executed to normalize the application. It receives the type of the axiom, the callee, and the arg of the application as parameters. The first two lines access the `world` in which the MimIR program graph is stored and destructure the `arg` tuple containing the natural number operands `a` and `b`. It then checks whether the operation is commutative and reorders its operands into a canonical order to simplify later comparisons. Following this, it performs the runtime check `Lit::isa()` to determine whether the operands are literals. If they are literals, the variables `la` and `lb` store their values. Then, the factory method `world.lit_nat()` is used to constant fold the literals `la` and `lb` into a new natural number literal. Otherwise, a non-normalizing application is created via `world.raw_app()` because the application cannot be normalized.

2.1.5 Phases

To facilitate domain-specific program transformations and the conversion between abstraction layers, MimIR closely integrates its plugin architecture with its phase infrastructure. In addition to declarations and normalizers, a plugin can be bundled with any number of phases implemented in C++ using MimIR’s phase [API](#). When optimizing a program, the MimIR compiler sequentially applies a series of phases in an order defined by a customizable compilation function. Each phase analyzes or transforms the program and passes it on to the next phase in the pipeline.

A typical workflow for a high-level plugin begins with phases that implement domain-specific transformations. Following this, the uses of the plugin’s axioms are converted to a set of lower-level axioms with a dedicated phase. MimIR provides the core, mem, and math plugins, which are intentionally modeled on LLVM’s instruction set. This enables MimIR’s LLVM backend to emit valid LLVM IR for MimIR programs that have been lowered in this way. (Leißa et al., 2025)

To facilitate the implementation of new phases, MimIR provides two abstract base classes. The `RWPhase` base class is used for phases whose main purpose is to transform a MimIR program, while the `Analysis` base class is used for phases that analyze the structure of a MimIR program without altering it. [Listing 2.12](#) demonstrates a rewrite phase that performs the same constant folding as the normalization function introduced in [Listing 2.11](#). This example illustrates an alternative mechanism for performing this program transformation, but in practice, the normalizer is responsible for it.

```

1  const Def* FoldAdd::rewrite_imm_App(const App* app) {
2      auto& w = new_world();
3
4      if (Axi::isa<core::nat::add>(app->callee())) {
5          auto [a, b] = app->arg()->projs<2>();
6          auto la = Lit::isa(a);
7          auto lb = Lit::isa(b);
8
9          if (la) {
10             if (lb) {
11                 return w.lit_nat(*la + *lb);
12             }
13         }
14     }
15
16     return RWPhase::rewrite_imm_App(app);
17 }

```

LISTING 2.12: Constant folding with a rewrite phase

In this example, we implement the `FoldAdd` phase, which inherits from the `RWPhase` base class. The `RWPhase` provides a rewrite method for every [IR](#) construct, whose implementation determines how the corresponding construct is rewritten when the phase is applied. In this case, we provide an implementation of the method `rewrite_imm_app()` to specify a program transformation for immutable applications in the MimIR program. The `RWPhase` also provides two worlds as class members that are accessible through `old_world()` and `new_world()`. They contain the program as it enters the phase and after it has been rewritten through the phase, respectively. The default behavior of any unimplemented rewrite method is to recreate the corresponding constructs in the new world unchanged.

The rewrite method begins by creating a reference `w` to the `new_world()`. It then checks at runtime whether the callee of the application is an instance of the `%core.nat.add_axiom` via `Axm::isa<core::nat::add>(app->callee())`. If this check is successful, it deconstructs the arguments of the application and stores them in variables `a` and `b`. As in Listing 2.11, it checks whether the arguments are literals and stores their values in the variables `la` and `lb`. If they are literals, it folds them into a new natural number literal with the factory method `w.lit_nat(*la + *lb)`. Otherwise, it calls the corresponding rewrite method of the `RWPhase` base class, which recreates the application unchanged in the new world.

2.2 Equality Saturation

Equality saturation is a compiler optimization technique first proposed by Tate et al. (2011). The subsequent introduction of `egg` by Willsey et al. (2021) has made this technique more accessible to projects related to compilers and theorem provers by providing a general-purpose library for integrating equality saturation into existing applications. Equality saturation attempts to solve a prominent problem in the field of compiler optimization known as the phase-ordering problem (Barthou and Touati, 2006).

In traditional compilers, optimizations are performed as a sequence of phases that are each responsible for performing a distinct program transformation. This process is destructive in the sense that a phase takes a program as its input, rewrites parts of it, and passes an altered version of the program to the next phase. An issue that arises from this approach is that altering the program in one phase may eliminate an optimization opportunity in a subsequent phase. Conversely, it is also possible that a potential program transformation is enabled only by a prior rewrite. It is therefore not trivial to define an optimal sequence of phases. This is encapsulated in the phase-ordering problem, which states that the order in which optimization phases are executed directly influences the quality of the generated code. (Tate et al., 2011)

$$\frac{x * n}{n} \rightarrow x \quad \text{if } n \neq 0 \quad (2.5)$$

$$x * 2^n \rightarrow x \ll n \quad (2.6)$$

We demonstrate the phase-ordering problem with a concrete example. Consider the common program transformations of strength reduction in (2.6) and inverse-elimination in (2.5). Both of these can be applied to the term $\frac{x*4}{4}$. The application of strength reduction produces the term $\frac{x \ll 2}{4}$ for which inverse-elimination is no longer applicable. On the other hand, applying inverse-elimination produces the smaller term x . This example shows that the order in which transformations are applied directly impacts the quality of the resulting code.

A potential solution to the phase-ordering problem consists of computing every possible rewrite sequence for a given set of program transformations. However, achieving such a solution with a traditional brute-force algorithm is not feasible because it results in space complexity that is exponential in the number of rewrites. (Willsey et al., 2021) The e-graph data structure that was first introduced by Nelson and Oppen (1980) enabled a new approach to exploring rewrite sequences. The e-graph provides a way to simultaneously represent many equivalent program terms in a shared structure, which facilitates the process of non-destructive rewriting that lies at the heart of equality saturation.

Equality saturation begins by populating an e-graph with an initial term. It proceeds to add equivalent terms via the application of program transformations (represented by rewrite rules) in a loop until the e-graph is saturated or a termination condition, for example, an iteration limit, is reached. Saturation in this context means that none of the considered transformations adds terms to the e-graph that are not already represented. Finally, a term is extracted from the e-graph according to a cost function that measures the quality of terms in relation to an optimization target. A commonly used cost function computes the size of a term from its **abstract syntax tree (AST)**. (Willsey et al., 2021)

In this section, we describe the symbolic expression format that bridges the gap between the program representations of different languages, including MimIR, and the input format expected by an equality saturation library. We also introduce the concept of rewrite rules and explain their role in expressing term equivalences. Furthermore, we introduce in greater detail the e-graph data structure, as well as the libraries that we used in our work. We explain the techniques of e-graph saturation, term extraction, and abstract interpretation that are commonly encountered in equality saturation, and present the corresponding abstractions that equality saturation libraries provide for these techniques.

2.2.1 Symbolic Expressions

Symbolic expressions (s-expressions) are a form of representation for tree-structured data. One of their earliest formal descriptions is found in the presentation of the Lisp programming language by McCarthy (1960). An important use case for s-expressions is the representation of source code with its underlying syntactic structure, as demonstrated in Lisp. They also constitute the input format expected by the equality saturation libraries that we used in our work. The grammar in (2.7) provides a formal definition of s-expressions. The terminal *Atom* describes an atomic symbol. We interpret the first *Atom* in an s-expression list as a function symbol. We interpret the following s-expressions as the arguments for that function.

$$\begin{aligned} \text{sexpr} &::= \text{Atom} \\ &\quad | (\text{sexprList}) \end{aligned} \tag{2.7}$$

$$\begin{aligned} \text{sexprList} &::= \text{sexpr} \\ &\quad | \text{sexpr } \text{sexprList} \end{aligned}$$

To demonstrate how terms can be represented in the s-expression format, (2.8) shows the conversion of the arithmetic expression $a * (b + c)$.

$$a * (b + c) \xrightarrow{\text{S-Expr}} (* \text{ a } (+ \text{ b } \text{ c})) \tag{2.8}$$

The tree-like structure of the term is captured in the nesting of s-expressions. The infix operators $*$ and $+$ are represented with corresponding function symbols that appear as the first elements in the s-expression lists. The first operand of the multiplication $a * (b + c)$ is modeled as the s-expression *a*, and its second operand is modeled as $(+ \text{ b } \text{ c})$. The latter s-expression represents the addition $(b + c)$.

The terms in the lambda calculus also follow a tree-like structure and can be represented similarly using s-expressions. We demonstrate the conversion of the

nested lambda abstraction $\lambda x.\lambda y. x y$ in (2.9).

$$\lambda x.\lambda y. x y \xrightarrow{\text{S-Expr}} (\text{lam } x (\text{lam } y (\text{app } x y))) \quad (2.9)$$

The lambda abstractions introducing the variables x and y are modeled as s-expressions with the function symbol `lam`. In their respective second positions, these s-expressions contain the atoms x and y that represent the introduced variables. Their third positions contain the s-expressions that model the bodies of the lambda abstractions. For the outer lambda, that is `(lam y (app x y))` and for the inner lambda, that is `(app x y)`. The application $x y$ is modeled as the s-expression `(app x y)`, where the second position is occupied by the callee x and the third position is occupied by the argument y .

MimIR is rooted in the **CoC**, a variant of the lambda calculus. The structural similarity between MimIR terms and the terms of the lambda calculus allowed us to model MimIR terms in a manner similar to the example introduced in (2.9). We illustrate the conversion of the term `lam foo(x: Nat) = (x, 1);` to its corresponding s-expression form in (2.10).

$$\text{lam foo}(x: \text{Nat}) = (x, 1); \xrightarrow{\text{S-Expr}} (\text{lam } x (\text{tuple } (\text{var } x) (\text{lit } 1))) \quad (2.10)$$

In this example, we define the lambda `foo` that introduces the variable x of type `Nat`. This lambda produces a tuple consisting of x and the natural number literal 1. The representation of lambda abstractions corresponds to the previous example. We represent the variable `foo` that is bound to the lambda expression with a let-binding `(let foo (lam ...))`, which we omitted in (2.10) for brevity. The s-expression `(tuple (var x) (lit 1))` models the tuple $(x, 1)$. We use the `var` function symbol to represent the occurrence of a variable and distinguish it from the introduction of that variable. The literal 1 is explicitly identified with a `lit` expression to distinguish it from a regular atom. Notably, the types of the variable x and the literal 1 are not represented. This is because we implemented distinct typed and untyped s-expression encodings that we use in different contexts. We introduce the format of typed s-expressions and their purpose in [subsection 3.2.3](#).

The use of s-expressions allowed us to represent MimIR terms in a format that is interpretable by the equality saturation libraries that we introduce in [subsection 2.2.4](#) and [subsection 2.2.5](#).

2.2.2 Rewrite Rules

Rewrite rules are a concept that originates in the area of theoretical computer science known as term rewriting. Term rewriting is a computational model that is rooted in equational logic. What distinguishes it from equational logic is its use of equations as directed rewrite rules. A rewrite rule expresses a directed rewriting relation between two terms consisting of variables, constant symbols, and function symbols. The application of a rewrite rule replaces a matching instance of its left-hand side term with an instance of its right-hand side term. Term rewriting is used in systems that require methods for reasoning with equations. (Baader and Nipkow, 1998) A particularly relevant example of a technique that incorporates term rewriting is equality saturation. We illustrate a minimal example of a rewrite rule in (2.11).

$$x + 0 \rightarrow x \quad (2.11)$$

This rewrite rule expresses the additive identity. The term $x + 0$ constitutes its left-hand side, where x is a variable, 0 is a constant symbol, and $+$ is a function symbol. The term x constitutes its right-hand side. The application of this rewrite rule to the concrete term $2 + 0$ yields the term 2 . More precisely, the left-hand side $x + 0$ matches the term $2 + 0$ under the substitution $\sigma = \{x \mapsto 2\}$. The same substitution is then applied to the right-hand side x to produce the term 2 .

The terms of a programming language such as MimIR often contain program variables that are different from the variables we introduced in (2.11). The variables of a rewrite rule, also called pattern variables, stand for arbitrary terms that are instantiated upon matching a concrete program term, whereas a program variable is a part of the program term being matched. Rewrite rules require a way to distinguish between pattern variables and program variables. A representation that achieves this distinction consists of a `?` prefix for pattern variables. This is also the representation chosen by the equality saturation libraries with which we worked. We demonstrate a concrete example of a rewrite rule for a MimIR term in (2.12).

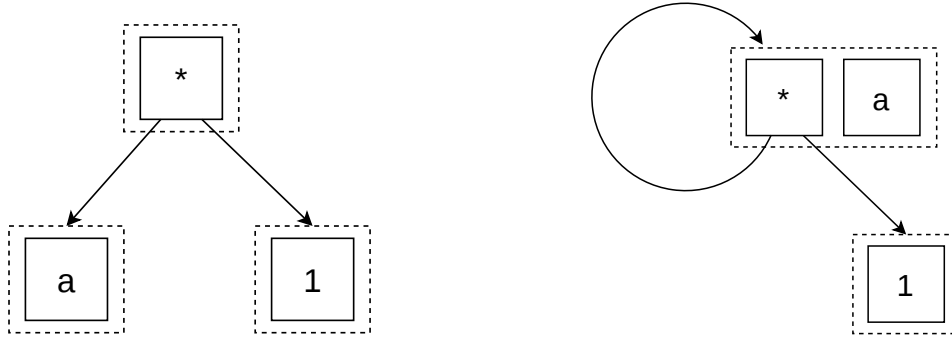
$$\text{\%core.nat.add } (?x, 0) \rightarrow ?x \quad (2.12)$$

This rewrite rule utilizes the axiom `\%core.nat.add` that we introduced in subsection 2.1.4 and the pattern variable `?x` to express the additive identity in MimIR. The application of this rule to the program term `\%core.nat.add (a, 0)` yields the term `a`. The left-hand side of the rewrite rule matches the term `\%core.nat.add (a, 0)` under the substitution $\sigma = \{?x \mapsto a\}$. Then, the rule produces the term `a` by applying σ to its right-hand side.

In equality saturation, rewrite rules are used to express the space of program transformations that should be explored by an equality saturation engine. In a conventional term rewriting system, this process of exploration incurs a significant computational cost because many possible rewrite sequences have to be explored and maintained separately. Equality saturation addresses this limitation by incorporating the e-graph data structure into its exploration procedure. (Willsey et al., 2021) In the following subsection, we describe the e-graph data structure in detail and elaborate the e-graph mechanisms that facilitate equality saturation.

2.2.3 E-Graphs

The e-graph (equality graph), initially proposed by Nelson and Oppen (1980), is a graph-based data structure that consists of a set of e-classes (equivalence classes). Each e-class contains a set of e-nodes that represent equivalent program terms of a given language. An e-node is described by a function symbol and a list of child e-classes. (Willsey et al., 2021) Unlike alternative data structures used to represent program terms, such as ASTs or directed acyclic graphs (DAGs), the e-graph is capable of simultaneously representing many equivalent terms. Figure 2.1a illustrates the representation of the arithmetic term $a * 1$ in an e-graph. Notably, the initial e-graph representing this term structurally resembles a regular DAG. The main difference lies in the dotted boxes surrounding each e-node, which represent the e-class containing the e-node. Figure 2.1b illustrates the same e-graph after applying the rewrite rule $x * 1 \rightarrow x$. Both e-nodes representing the terms $a * 1$ and a are now contained in the same e-class. This shared representation of terms before and after rewriting directly enables the process of non-destructive rewriting that underlies equality saturation.

(A) E-graph representing the term $a * 1$ (B) E-graph after applying the rewrite rule $x * 1 \rightarrow x$

E-Matching

The mechanism of finding matches for rewrite patterns in an e-graph, also called e-matching, is similar to the procedure that we described previously in [subsection 2.2.2](#). However, instead of finding a single substitution σ , e-matching searches for tuples (σ, c) . Every e-class in an e-graph is represented by an e-class ID. The tuple (σ, c) consists of a substitution σ that maps the rule's pattern variables to e-class IDs. The e-class identified by c represents the left-hand side of the rule under σ . Upon application, the right-hand side under σ is instantiated and added to the e-graph. If the term being added is not yet represented in an e-class, a new e-class is created for it; otherwise, the existing e-class is reused. As the two terms have been established as equivalent, their corresponding e-classes are merged into one e-class. (Willsey et al., 2021)

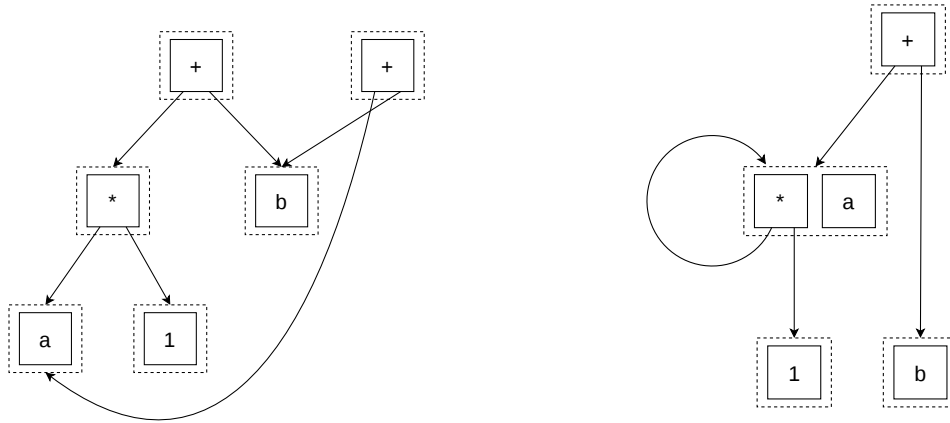
Suppose that in [Figure 2.1a](#), the e-classes are associated with e-class IDs by the following mapping: $\{a\} \mapsto c_a$, $\{1\} \mapsto c_1$, and $\{*(c_a, c_1)\} \mapsto c_*$. E-matching the rule $x * 1 \rightarrow x$ in the e-graph finds a tuple $(\sigma = \{x \mapsto c_a\}, c_*)$. Adding the instantiated right-hand side does not modify the e-graph as it is already represented. Finally, the e-classes $\{a\}$ and $\{*(c_a, c_1)\}$ are merged into one e-class $\{a, *(c_a, c_1)\}$.

Congruence Closure

In addition to representing groups of equivalent terms, e-graphs also maintain a congruence relation over those terms. The congruence relation encapsulates the property that two e-nodes are congruent if they consist of the same function symbol and their corresponding child e-classes are equivalent. This relation is formally described by (2.13). Here, a_i and b_i denote the child e-class IDs of two e-nodes with function symbol f . Equivalence between e-class IDs and congruence between e-nodes are indicated by $\equiv$ and $\cong$, respectively.

$$a_1 \equiv b_1, \dots, a_n \equiv b_n \implies f(a_1, \dots, a_n) \cong f(b_1, \dots, b_n) \quad (2.13)$$

The e-graph maintains a congruence closure, which guarantees that the equivalences implied by the congruence relation always hold. In practice, the congruence closure ensures that merging e-classes triggers further merges between e-classes containing newly congruent e-nodes. We illustrate this process with a concrete example in [Figure 2.2a](#) and [Figure 2.2b](#).



(A) E-graph representing the terms $a * 1 + b$ and $a + b$

(B) E-graph after applying the rewrite rule $x * 1 \rightarrow x$ and restoring congruence

We begin by constructing an e-graph that represents the terms $a * 1 + b$ and $a + b$ in Figure 2.2a. These terms are initially represented in different e-classes. After applying the rewrite rule $x * 1 \rightarrow x$, the e-classes representing the terms $a * 1$ and a merge together. The congruence closure ensures that the e-nodes representing the terms $a * 1 + b$ and $a + b$, which have become congruent as a result of the merge, are also represented in the same e-class. The resulting e-graph is shown in Figure 2.2b.

In the preceding subsections, we have introduced the most relevant mechanisms behind equality saturation. Rewrite rules specify the space of program transformations that an equality saturation engine should explore. E-graphs offer a shared representation of term equivalences discovered through the repeated e-matching and application of rewrite rules. Finally, the congruence closure ensures that these equivalences are propagated through the e-graph. In the following subsection, we introduce the egg (Willsey et al., 2021) library, which incorporates all of these concepts to facilitate the integration of equality saturation into existing applications.

2.2.4 Egg

Equality saturation is an optimization technique that is not trivial to integrate into existing applications. Several specialized components, such as the e-graph data structure, e-matching, e-graph saturation, and term extraction, have to be implemented from scratch or integrated into the application. Egg is a general-purpose library developed by Willsey et al. (2021) that addresses this challenge by providing a language-agnostic framework for equality saturation. It provides implementations for all of the aforementioned components while remaining generic over a user-defined term language. In this subsection, we introduce the abstractions that egg provides for different equality saturation mechanisms and demonstrate their use on a concrete example.

Language Definition

Egg provides an equality saturation engine capable of operating on the terms of a user-defined language. This is achieved with the `define_language!` macro, which

facilitates the definition of language operations and primitives. The resulting language definition allows egg to parse terms represented as s-expressions into its internal recursive expression representation, `RecExpr<L>`. The parsed language terms are then used in the construction of e-graphs. The patterns of rewrite rules are also defined over the same language.

```

1  define_language! {
2      enum Arith {
3          "+" = Add([Id; 2]),
4          "-" = Sub([Id; 2]),
5          "*" = Mul([Id; 2]),
6          "/" = Div([Id; 2]),
7          Var(String),
8          Num(i32),
9      }
10 }
11
12 let example: RecExpr<Arith> = "(* a (+ 1 0)).parse().unwrap();

```

LISTING 2.13: Definition of an arithmetic language using egg's `define_language!` macro followed by a term parsed as `RecExpr<Arith>`

Here, we provide a definition of the arithmetic expression language `Arith`. This language consists of numeric literals, variable symbols, and binary operations with function symbols `+`, `-`, `*`, and `/`. We provide a definition of the binary operations in the first four lines of the `Arith` enum. More precisely, we assign a language variant to a string representation of its function symbol as in `"+" = Add([Id; 2])`. This string representation is used to associate an s-expression with its corresponding internal representation during parsing. In the context of e-graphs, this language variant describes an e-node with the function symbol `+` and two child e-class IDs represented in the array `[Id; 2]`. The variable symbols and numeric literals are modeled with the language variants `Var(String)` and `Num(i32)`, respectively. These variants carry data of types `String` and `i32`, whereas the previous operations only carry child `Ids`.

Following the definition of the `Arith` language, we define the arithmetic term $a * (1 + 0)$ as the s-expression `(* a (+ 1 0))`. The function symbols `+` and `*` used in this s-expression correspond to the language variants `Add` and `Mul`. The variable `a` and the numeric literals `1` and `0` correspond to the language variants `Var` and `Num`. We proceed to parse the s-expression into egg's internal representation, `RecExpr<Arith>`. This recursive expression represents the same term as a list of language nodes that incrementally build upon previous nodes. The last node in this list is the root of the expression. We previously explained that in the context of an e-graph, a language variant describes an e-node with child e-class `Ids`. In the context of a recursive expression, the child `Ids` refer to prior indices in the recursive expression list. The result of parsing the s-expression is illustrated in [Table 2.1](#).

Id	Language Node
0	Var("a")
1	Num(1)
2	Num(0)
3	Add([1, 2])
4	Mul([0, 3])

TABLE 2.1: Internal RecExpr<Arith> representation of the term $a * (1 + 0)$

Here, the beginning of the list contains representations of the variable a and the numeric literals 1 and 0 in the respective indices 0, 1, and 2. The following `Add([1, 2])` node refers back to the representations of 1 and 0 by their respective indices. The `Mul([0, 3])` node, which is the root of the expression, is built on the prior nodes `Var("a")` and `Add([1, 2])`.

Rewrite Rules

To define a space of program transformations for the terms of the previously defined language, egg provides the `rewrite!` macro. This macro facilitates the definition of syntactic rewrite rules, which we introduced in [subsection 2.2.2](#). In addition, egg's rewrite API further enables the definition of conditional and dynamic rewrite rules. We discuss the purpose and construction of such rewrite rules later in this subsection. [Listing 2.14](#) shows how we used the `rewrite!` macro to define rules for the additive and multiplicative identities.

```
1 let rules = vec![
2   rewrite!("add-0"; "(+ ?x 0)" => "?x"),
3   rewrite!("mul-1"; "(* ?x 1)" => "?x"),
4 ]
```

LISTING 2.14: Definition of the rewrite rules $x + 0 \rightarrow x$ and $x * 1 \rightarrow x$ in egg

Here, we define the two rewrite rules named `add-0` and `mul-1` and store their definitions in the `rules` variable for later use. The `rewrite!` macro constructs a new rewrite rule from a given name and two s-expressions representing the respective left- and right-hand side patterns of the rule. We represent the left-hand side of the additive identity $x + 0$ as the s-expression `(+ ?x 0)` and its right-hand side x as the s-expression `?x`. The pattern variable x is denoted with `?x` following the convention we introduced in [subsection 2.2.2](#). We proceed in a similar way for the multiplicative identity.

E-Graph Saturation

Following the specification of rewrite rules, an equality saturation engine explores the space of program transformations through repeated e-matching and application of these rules in an e-graph. Egg provides the `Runner` abstraction that is responsible for initializing an e-graph from an input term and then attempting to saturate the e-graph through the iterative application of the provided rules. As we explained in previously, the saturation loop terminates when the e-graph is saturated or a configured limit has been reached. We demonstrate the construction of a `Runner` for the example term that we defined in [Listing 2.13](#).

```

1 let runner = Runner::default()
2     .with_expr(&example)
3     .run(&rules);

```

LISTING 2.15: E-graph saturation using egg's Runner

Here, we construct a Runner with egg's default configuration using `Runner::default()`. The method `with_expr(&example)` specifies the input term as the example term $a * (1 + 0)$, whose s-expression representation we parsed in Listing 2.13. The Runner then initializes an e-graph representing that term. Finally, `run(&rules)` attempts to saturate this e-graph with the rules defined in Listing 2.14. The resulting e-graph is accessible as a public field of the Runner struct.

Term Extraction

After the saturation loop has ended, an equality saturation engine has to extract a term from the e-graph that is equivalent to the given input term. The e-class representing the input term consists of the set of represented terms that are equivalent to it. This limits the choice of terms to the e-class representing the input term. Furthermore, the term should fulfill some criterion of optimality, which is commonly specified through a cost function. The cost function defines how the cost of an e-node is computed from the costs of its children. An extraction algorithm, for which egg provides the `Extractor` abstraction, then uses these costs to determine the least costly term represented in a given e-class. Commonly used cost functions include the size or depth of the AST representing a term. More complex cost functions may also assign costs to specific operations that appear in a term.

```

1 let example_eclass = runner.roots[0];
2 let extractor = Extractor::new(&runner.egraph, AstSize);
3 let (best_cost, best_term) = extractor.find_best(example_eclass);
4
5 assert_eq!(best_term.to_string(), "a");
6 assert_eq!(best_cost, 1);

```

LISTING 2.16: Term extraction using egg's Extractor with the AstSize cost function

Here, we initialize the variable `example_eclass` with the ID of the e-class representing the example term of Listing 2.13 by accessing `runner.roots[0]`. The roots of a Runner store the e-class IDs associated with the terms with which it initialized its e-graph in the order in which they were added to the e-graph. Following this, we instantiate a new `Extractor` by providing it with the e-graph of the Runner and the `AstSize` cost function. The `AstSize` cost function is provided by egg and assigns a cost based on the number of AST nodes of a represented term. After saturation, the e-class representing the term $a * (1 + 0)$ also represents the terms $a * 1$ and a . The respective costs of these terms according to `AstSize` are 5, 3, and 1. Finally, we call the `Extractor`'s method `find_best(example_eclass)` to extract the least costly term `best_term` represented in the `example_eclass` along with its cost `best_cost`. We assert that the extracted term is equal to the term a and that its cost is equal to 1.

E-Class Analysis

In the preceding subsections, we have discussed the different mechanisms with which an equality saturation engine explores a space of program transformations using syntactic rewrite rules. In practice, many forms of program optimization rely on more than just syntactic information about an input program. Semantic facts, such as free variables, constant values, or type information, are often taken into account when transforming an input term. E-class analysis is a technique devised by Willsey et al. (2021) that aims to facilitate the computation of semantic facts about the terms represented in an e-graph. The theoretical foundation of e-class analysis lies in the area of computer science known as abstract interpretation. Abstract interpretation is a technique that allows a compiler to reason about the properties of concrete programs by modeling their execution in an abstract domain. Information in this domain can be combined and propagated without requiring detailed knowledge of the program’s concrete behavior. (Cousot and Cousot, 1977)

The e-classes of an e-graph are associated with analysis data which describes the represented terms in an abstract domain. This domain, together with a join operation, forms a join-semilattice, which means that its values are partially ordered, and any pair of values can be combined according to their least upper bound. The join operation defines how analysis data is combined with the merging of e-classes.

The analysis interface consists of the methods `make`, `merge`, and `modify`. The `make` operation, which is analogous to an abstraction function, constructs analysis data from a newly added e-node. It typically achieves this by accessing the analysis data associated with the new e-node’s child e-classes. The `merge` operation defines how the data associated with merging e-classes is joined together. Lastly, the `modify` operation modifies an e-class based on its associated analysis data. In cases where this leads to a direct representation of the analysis data in the form of a new e-node, this operation is akin to a concretization function. (Willsey et al., 2021)

A typical use case for e-class analysis is constant folding, which we demonstrate in Listing 2.17, but it can also be used for more complex tasks. In subsection 3.4.6, we explain how we maintained the type information of MimIR program terms using e-class analysis.

```

1  struct ConstantFolding;
2  impl Analysis for ConstantFolding {
3      type Data = Option<Num>;
4
5      fn make(egraph: &EGraph, enode: &Arith) -> Self::Data {
6          let c = |id: &Id| egraph[*id].data.clone();
7          match enode {
8              Num(_) => Some(enode.clone()),
9              Add([x, y]) => Some(Num(c(x)? + c(y)?)),
10             Sub([x, y]) => Some(Num(c(x)? - c(y)?)),
11             Mul([x, y]) => Some(Num(c(x)? * c(y)?)),
12             Div([x, y]) => Some(Num(c(x)? / c(y)?)),
13             _ => None,
14         }
15     }
16     // ...
17 }
```

LISTING 2.17: `make` function of the ConstantFolding Analysis

Listing 2.17 shows the `ConstantFolding` analysis which performs constant folding over the terms of the previously defined language `Arith`. We define the domain of the analysis data as the type `Option<Num>` to associate e-classes with an optional constant value represented as a `Num` language variant. Following this, we define the `make` function, whose parameters consist of the egraph and the newly added enode. We match this enode against the existing language variants and construct a new `Num` that represents the constant value associated with its e-class. For example, an enode `Add([x, y])` produces a `Num` that represents the result of adding together the constants associated with e-classes `x` and `y`.

```

1  struct ConstantFolding;
2  impl Analysis for ConstantFolding {
3      // ...
4      fn merge(left: &mut Self::Data, right: Self::Data) -> DidMerge {
5          match (left, right) {
6              (None, None)      => DidMerge(false, false),
7              (Some(_), None)   => DidMerge(false, true),
8              (None, Some(c))    => { *left = Some(c); DidMerge(true, false) }
9              (Some(c1), Some(c2)) => { assert!( c1 == c2 ); DidMerge(false, false) }
10         }
11     }
12 }

```

LISTING 2.18: merge function of the `ConstantFolding` Analysis

Listing 2.18 shows the merge function of the `ConstantFolding` Analysis, whose parameters consist of the data associated with the two e-classes being merged. The `right` value merges into the `left`, which represents the joined analysis data of the merged e-class. This is also the reason why the `left` parameter is a mutable reference. If `left` contains no constant and `right` does contain a constant, which is expressed with the match arm `(None, Some(c))`, this constant is assigned to the `left`. If `left` already contains a constant and `right` does not, the `left` constant is retained. Finally, if both e-classes are already associated with constants, an assertion checks the invariant of our `ConstantFolding` Analysis that the constant values associated with merging e-classes must be equal. This is because they were established as equivalent prior to the merge. The Boolean values in `DidMerge` indicate whether the joined analysis data differs from the respective original `left` and `right` values.

A complete implementation of the `ConstantFolding` Analysis would further contain a definition of a `modify` function, which extends the e-graph with concrete representations of the computed constant values. We omit this implementation for the sake of keeping this subsection concise.

Conditional Rewrite Rules

In the previous subsections, we have introduced syntactic rewrite rules and explained how an equality saturation engine uses them to explore a transformation space. In some cases, expressing a program transformation requires rewrite rules whose application takes into account previously computed analysis data. Egg distinguishes between conditional and dynamic rewrite rules, where the former are a subset of the latter. Conditional rewrite rules are syntactic and apply only after a successful check involving analysis data. Dynamic rewrite rules utilize egg's rewrite **API** to dynamically compute a right-hand side based on existing analysis data. (Willsey et al., 2021) We limit the explanations in this subsection to conditional

rewrite rules, since our work relied only on such rewrite rules. [Listing 2.19](#) demonstrates the definition of the conditional rewrite rule $x/x \rightarrow 1$ if $x \neq 0$ in egg.

```
1 let div_elim = rewrite!(
2   "div-elim";
3   "(/ ?x ?x)" => "1"
4   if is_nonzero("?x")
5 );
```

 Rust

LISTING 2.19: A conditional rewrite rule $x/x \rightarrow 1$ if $x \neq 0$ expressed with egg’s `rewrite!` macro

The `rewrite!` macro, which we used earlier to define regular syntactic rewrite rules, is also capable of defining conditional rewrite rules. We represent the respective left- and right-hand side patterns of the rule with the s-expressions `(/ ?x ?x)` and `1`. In addition, we define the rule’s condition $x \neq 0$ through `is_nonzero("?x")`, whose implementation we omit for brevity. The function `is_nonzero` accesses the optional constant value associated with the e-class matched by the pattern variable `?x`. It returns a Boolean value `true` to indicate that the e-class is associated with a constant whose value is not equal to zero. This constant information is maintained by the previously defined `ConstantFolding Analysis`.

In this subsection, we have introduced the egg library and demonstrated the various abstractions it provides for different equality saturation mechanisms. Egg is capable of representing and rewriting the terms of many languages, but it faces a particular set of difficulties when dealing with the terms of a lambda calculus. These difficulties arise from representing and manipulating terms with binders and factoring α -equivalence into its notion of term equivalence. In the following subsection, we present the slotted-egraphs (Schneider et al., 2025) library and explain how it addresses these limitations.

2.2.5 Slotted E-Graphs

In the previous subsection, we briefly outlined the difficulties that arise when attempting to incorporate α -equivalence and binder-manipulating rewrites for lambda-calculus terms. In this subsection, we explain these challenges in detail with a particular focus on those associated with the named representation of variables, as this is the approach we utilized in our implementation of MimIR in egg. We then introduce the slotted-egraphs library developed by Schneider et al. (2025) and explain how it addresses these challenges. Since the [API](#) and abstractions exposed by this library closely resemble those of egg, we refrain from reintroducing them in this subsection.

Limitations of Named Variable Representations

Alpha-equivalence In a lambda calculus, two terms are considered α -equivalent if they represent the same term modulo variable namings. An example of two α -equivalent terms is the lambdas that introduce variables x and y in (2.14). Consistently renaming the variables x and y does not change their binding structure.

$$\lambda x. \lambda y. x y \equiv_{\alpha} \lambda y. \lambda x. y x \quad (2.14)$$

Since these terms are equivalent, an e-graph representing them should ideally group them into the same e-class. Egg’s notion of equivalence, however, is purely syntactic and does not inherently include equivalence modulo α -renaming. The result of this

is a loss of shared representation in the e-graph, which is one of the main driving forces behind equality saturation. This can lead to larger e-graphs as a result of redundant representations of α -equivalent terms. Figure 2.3 illustrates an e-graph representing the lambdas in (2.14).

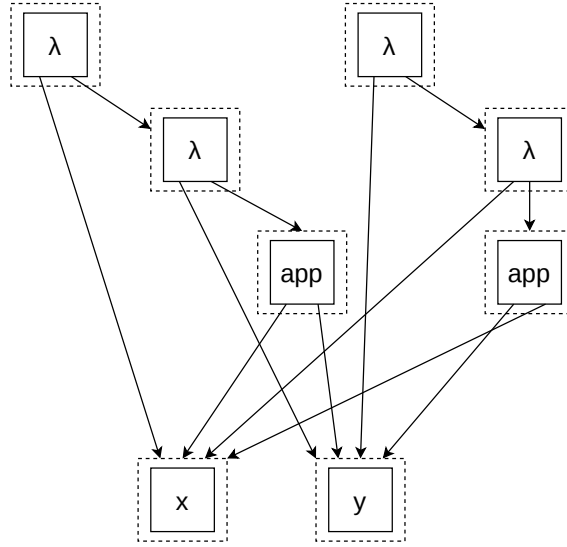


FIGURE 2.3: Distinct representations of the α -equivalent terms $\lambda x. \lambda y. x y$ and $\lambda y. \lambda x. y x$ in an e-graph

A well-established approach to obtaining a common representation of α -equivalent terms is to represent bound variables in a de Bruijn encoding (de Bruijn, 1972) in which they are represented by indices according to the relative distance from their binder. A de Bruijn encoding of the α -equivalent terms in (2.14) is shown in (2.15).

$$\lambda. \lambda. \hat{1} \hat{0} \quad (2.15)$$

Here, the occurrences of bound variables are represented by the indices $\hat{0}$ and $\hat{1}$, where $\hat{0}$ refers to the nearest enclosing binder and $\hat{1}$ refers to the binder one level above it. This de Bruijn encoding results in a unified representation of the previously distinct terms in (2.14). Moving this term into an e-graph results in a shared structural representation of the α -equivalent terms. Although this encoding facilitates the representation of α -equivalent terms, it comes with a new set of challenges. Rewrite rules that modify the binder structure of a term have to dynamically shift de Bruijn indices to account for changes in the relative distances to their respective binders.

Fresh binder generation In our egg implementation, we took advantage of the fact that MimIR assigns globally unique identifiers to its variables. Variables are uniquely identified by an integer suffix that represents their global ID. Two variables with identical surface names x may be internally represented as x_1 and x_2 . A challenge that arises with this globally unique named representation lies in the requirement of consistently generating fresh names in rewrite rules introducing new variables. Consider the rewrite rule in (2.16) that expresses η -expansion.

$$f \rightarrow \lambda x. f x \quad (2.16)$$

This rewrite requires a mechanism that generates a fresh unique identifier with every application. A purely syntactic rewrite rule is unable to implement such a mechanism, as its transformation does not incorporate an external state. In practice, this transformation can be achieved with a counter variable that stores the next available global ID. A dynamic rewrite rule then constructs the newly introduced variable x with a fresh global ID suffix obtained from the counter and finally increments that counter.

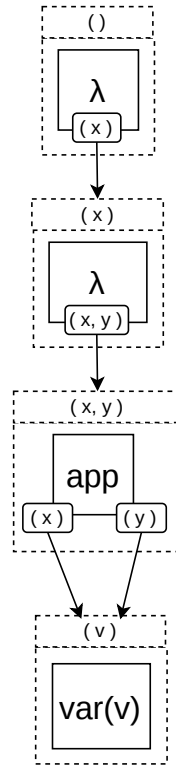
Binder-sensitive rewrite rules The validity of certain rewrite rules that manipulate binders directly depends on free variable information of the matched terms. This free variable information is typically maintained as part of an e-class analysis. Consider the conditional rewrite rule in (4.3) that expresses η -reduction.

$$\lambda x.f x \rightarrow f \quad \text{if } x \notin FV(f) \quad (2.17)$$

In this example, the function represented by f , whose surrounding binder structure the rewrite rule tries to naively eliminate, may itself depend on the variable x . A concrete instance of f may be the application $g x$ where x appears as a free variable. In this case, the rule would transform the term $\lambda x.(g x) x$ into the term $g x$. This leaves the occurrence of x free in the latter term, while x was originally bound in the former term. The conditional check $x \notin FV(f)$ ensures that this rule only applies when x is not contained in the free variables of f .

Slotted E-Graph Representation

A solution to the challenges we introduced earlier is presented in the concept of slotted e-graphs. The fundamental idea behind slotted e-graphs is to parametrize e-classes by the free variables of the terms that they represent. The input parameters of an e-class are called slots and the e-nodes they contain utilize these slots to invoke their child e-classes. The result of this is that the representation of a term captures its structure but is not constrained by the concrete names of the free variables that occur in it. This leads to an extension of the notion of equivalence in slotted e-graphs to also include equivalence modulo α -renaming. (Schneider et al., 2025) In addition, slotted e-graphs allow for the representation of terms that bind slots, which facilitates the representation of lambda abstractions. The challenge of generating fresh variable names is also mitigated as newly introduced binders are represented in a name-independent manner. Finally, since the slots of an e-class directly model the free variables of its represented terms, binder-sensitive rewrite rules no longer require a free variable analysis to check the free variables of a matched term. [Figure 2.4](#) illustrates a shared representation of the α -equivalent lambdas of (2.14) in a slotted e-graph.

FIGURE 2.4: The term $\lambda x.\lambda y.xy$ represented in a slotted e-graph

The singleton e-class that contains the `var` e-node represents single-variable terms such as x and y and abstracts their concrete names through the input slot v . This e-class is invoked by the `app` e-node contained in the e-class above that represents the application xy . This term contains two free variables, which is why its e-class requires the two slots x and y . Each variable in the application is represented with an invocation of the single-variable e-class. The term $\lambda y.xy$ is represented in the e-class above, which contains an e-node representing a lambda abstraction that binds the slot y . The term $\lambda x.\lambda y.xy$ is represented similarly in the e-class at the top.

As the authors of slotted-egraphs have shown, optimizations of lambda terms that heavily relied on binder manipulation significantly benefited from the use of a slotted e-graph in terms of runtime and memory efficiency. (Schneider et al., 2025) In our work, we implemented a plugin capable of optimizing MimIR terms with equality saturation using egg and slotted-egraphs, and we evaluate it on concrete RISE program transformations in [chapter 4](#).

Chapter 3

Implementation

In this chapter, we present our implementation of an equality saturation plugin for MimIR. We begin with an overview of the high-level workflow involved in optimizing a MimIR program through equality saturation. We then introduce the high-level abstractions exposed by the plugin that facilitate interactions with an equality saturation engine directly from MimIR. These abstractions are in the form of a set of axioms that orchestrate the egg and slotted-egraphs backends. We also introduce the configuration mechanisms of the plugin, and explain in detail the representation of MimIR in egg and slotted-egraphs. Finally, we demonstrate how the terms that were optimized by an equality saturation engine are reconstructed as MimIR terms.

3.1 Overview

In the following sections, we present in detail the steps that are involved in optimizing a MimIR program via equality saturation. We preface these sections with a high-level overview of this procedure. [Figure 3.1](#) illustrates the steps that a MimIR program undergoes until it reaches an optimized state.

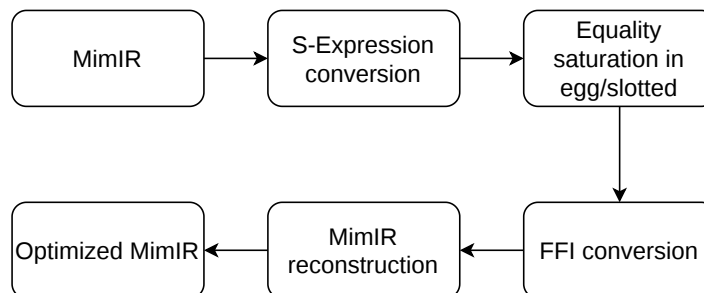


FIGURE 3.1: High-level workflow for optimizing a MimIR program through equality saturation

The first step that an unoptimized MimIR program undergoes is a conversion into s-expression format to become interpretable by the equality saturation backends implemented in egg and slotted-egraphs. Then, the equality saturation backends optimize the input program, taking into account various configuration parameters embedded in the input program as [IR](#) constructs of the equality saturation plugin. At this point, the optimized program is represented with recursive expressions in the Rust libraries egg and slotted-egraphs. In the following step, we map this internal representation to a shared [foreign function interface \(FFI\)](#) representation that facilitates interactions with the optimized program via MimIR's C++ plugin code. Finally,

the representation of the optimized program is reconstructed in MimIR through a rewrite phase of the equality saturation plugin.

3.2 MimIR as Symbolic Expressions

The first step a MimIR program undergoes during optimization is a conversion to the s-expression format. The purpose of this is to enable the egg and slotted-egraphs backends to parse and then rewrite the program via equality saturation. In this section, we demonstrate the conversion of MimIR programs into s-expressions with three concrete examples. The first example shows the conversion of core IR constructs, while the second example demonstrates how we encoded the nesting structure of a MimIR program in its s-expression representation. Finally, the third example shows an s-expression representation that additionally includes type annotations omitted in the first two examples.

3.2.1 Basic Symbolic Expression Representation

To facilitate the conversion of MimIR into the s-expression format, we implemented an s-expression emitter and directly integrated it into the MimIR compiler. This emitter recursively traverses an existing MimIR program and textually emits it as a series of s-expressions representing the roots of the program. Each s-expression in this series corresponds to one closed program term.¹ Typically, this includes every external term and any closed terms that are transitively reachable through an external term. We omit the implementation details of the s-expression emitter to maintain a reasonable scope for this subsection. Listing 3.1 shows a Mim program whose symbolic expression representation is shown in Listing 3.3.

```
1 fun extern foo(t: [Nat, Bool], i: Idx 2): Nat u Bool =
2   let res = t#i;
3   return res;
```

LISTING 3.1: Example function `foo` defined in Mim

Here, we define the external function `foo` with input parameters `t` and `i`, where `t` represents a tuple that contains a natural number and a Boolean, and `i` can be used to index `t`. The codomain of `foo` is the union type $\text{Nat} \cup \text{Bool}$, which indicates that `foo` may return a natural number or a Boolean. In the first line, the variable `res` is initialized with the result of extracting an element from `t` at the index `i`. Finally, `res` is returned.

```
1 lam extern foo([t: [Nat, Bool], i: Idx 2], return: Cn Nat u Bool)@ff: 1 =
2   let res = t#i;
3   return res;
```

LISTING 3.2: Desugared representation of `foo`

Listing 3.2 shows the same function `foo` from Listing 3.1 after it has been desugared into a corresponding lambda. We show this listing to emphasize the internal MimIR representation of `foo`, which forms the basis of our s-expression representation.

¹A closed term in a lambda calculus is a term that contains no free variables.

```

1  (root extern foo
2    (fun _12345 (lit ff)
3      (let return
4        (extract (var _12345) (lit tt))
5      (let t
6        (extract (extract (var _12345) (lit ff)) (lit ff))
7      (let i
8        (extract (extract (var _12345) (lit ff)) (lit tt))
9      (let res
10        (extract (var t) (var i))
11        (app (var return) (var res))))))

```

LISTING 3.3: S-expression representation of `foo` for the egg backend

Listing 3.3 shows the s-expression representation of `foo`. Every program root is wrapped in a root binder that stores its variable name and external attribute. We defined root binders as a special form of `let` binders to facilitate distinguishing the program roots and to more closely resemble MimIR’s surface syntax.

The lambda that is bound to `foo` is represented in the s-expression with the function symbol `fun`. The first position of this s-expression is occupied by the anonymous variable `_12345`, which represents the domain $[[t:\text{Nat}, \text{Bool}], i:\text{Idx } 2], \text{return}:\text{Cn Nat} \cup \text{Bool}]$ of `foo`. Its second position represents the lambda’s partial evaluation filter `ff`, and its last position represents its body.

The body of the lambda consists of four nested `let` bindings that define the respective variables `return`, `t`, `i`, and `res`. An s-expression with the function symbol `let` defines the variable name that it binds in its first position. Its second position is occupied by the expression bound to the variable name. Finally, its third position represents the expression in which the binding is available. Notably, the `res` variable can be bound to an expression using the variables `t` and `i` that were bound at an outer level. The s-expression `(app (var return) (var res))`, which represents the function body `return res`, can similarly use the variables `return` and `res` that were defined at an outer level. As we explained in [subsection 2.1.2](#), an s-expression with function symbol `var` distinguishes a variable use from its introduction.

A closer inspection of the expressions bound to the variables `return`, `t`, and `i` reveals various extractions from `foo`’s domain, which is represented by the variable `_12345`. This variable represents a two-element tuple that contains another two-element tuple consisting of `t` and `i` in its first position and the `return` continuation in its second position. To retrieve, for example, the `return` continuation, the second element of `_12345` has to be extracted. This extraction is represented by the s-expression `(extract (var _12345) (lit tt))`. The reason why the extraction index is the literal `tt` is that `tt` is an alias for the literal `1_2` as we explained in [subsection 2.1.2](#).

The slotted-egraphs library required a slightly different s-expression representation from the one we introduced above. In particular, we prefixed all variable names bound by a non-root binder with a `$` symbol to mark them as slots. The resulting symbolic expression is shown in [Listing 3.4](#).

```

1  (root extern foo
2    (fun $_12345 (lit ff)
3      (let $return
4        (extract (var $_12345) (lit tt))
5      (let $t
6        (extract (extract (var $_12345) (lit ff)) (lit ff))
7      (let $i
8        (extract (extract (var $_12345) (lit ff)) (lit tt))
9      (let $res
10        (extract (var $t) (var $i))
11        (app (var $return) (var $res))))))))

```

LISTING 3.4: S-expression representation of foo for the slotted-egraphs backend

3.2.2 Encoding Lambda Nesting

Many root terms in MimIR are equipped with a control flow structure that is expressed through the nesting of continuations, as we have shown in the minimal example in Listing 2.5. This structure is explicitly represented in the Mim surface syntax, where the `where..end` construct encapsulates the definitions of the nested continuations. However, MimIR uses an internal representation modeled on the concept of a sea-of-nodes graph where every expression is represented in the same graph. The result of this is that the nesting structure is no longer explicitly represented. MimIR addresses this with an additional graph structure that it calls the nesting tree. The nesting tree is computed from the free variables of lambdas and captures their nesting structure. To illustrate this with a concrete example, we introduce a nested recursive function in Listing 3.5.

```

1  plugin core;
2
3  fun extern main(start: Nat, ending: Nat) =
4    loop start
5    where
6      con loop(n: Nat) =
7        let cond = %core.ncmp.lt (n, ending);
8        (exit, body)#cond ();
9      where
10        con body() =
11          let inc = %core.nat.add (n, 1);
12          loop inc;
13        con exit() =
14          return n;
15      end
16    end

```

LISTING 3.5: Mim representation of the main function with nested continuations

Here, we define the external main function with two natural number input parameters `start` and `ending`. In its body, it applies the `loop` continuation, whose definition follows in the next `where..end` block, to the `start` parameter. The `loop` continuation is nested within the main function and takes the natural number parameter `n`. It initializes the variable `cond` with the result of comparing `n` with the parameter `ending` of the main function. Notably, `ending` appears as a free variable in

the loop continuation. It then uses `cond` to extract one of the two continuations `exit` and `body` from a tuple, and applies the extracted continuation to an empty tuple. This is akin to a conditional branch, similar to the example introduced in Listing 2.5.

Finally, the continuations `body` and `exit` are nested within the loop continuation. These are analogous to basic blocks that express the semantics of the body and exit of a loop. The `body` increments the parameter `n` of the loop continuation and then applies `loop` again to the incremented value `inc`. The `exit` continuation applies the main function’s return continuation to return the parameter `n`. In both of these continuations, `n` occurs as a free variable, which explains why they are nested within the loop continuation.

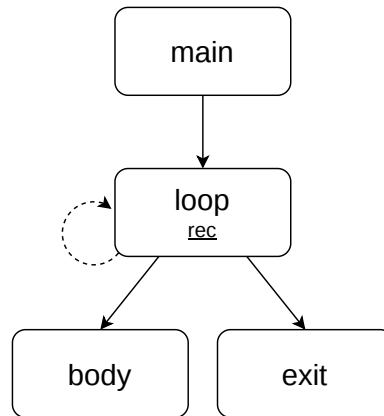


FIGURE 3.2: Nesting tree computed for the main function

The nesting tree of the main function is shown in Figure 3.2. It directly reflects the layers of nesting that were expressed in `where . . end` blocks in Mim. We reiterate that this graph is computed from MimIR’s internal graph representation and therefore reconstructs a nesting structure that is not explicitly retained there. The loop continuation is additionally marked as recursive because it applies the body continuation, which itself applies the loop continuation.

MimIR provides an `Emitter` base class for the purpose of implementing compiler backends for MimIR programs. This class serves, among others, as the basis for MimIR’s LLVM IR emitter. The `Emitter` transitively visits all closed terms and computes their nesting trees. As the `Emitter` traverses each visited term, it utilizes the nesting tree to determine the order in which terms are emitted. For our s-expression backend, we relied on the `Emitter` to embed the nesting structure into the representation of root terms. This encoding became particularly relevant when reconstructing optimized terms as MimIR, as we explain in section 3.5. The s-expression representation of the main function is shown in Listing 3.6. To keep the example concise, we omitted the lambda bodies.

```

1  (root extern main
2    (fun _2030 (lit ff)
3      (let loop_2071
4        (con _2072 (lit ff)
5          (let exit_2110
6            (con _2111 (lit ff)
7              ...
8            (let body_2124
9              (con _2125 (lit ff)
10                ...
11              ...
12            ...

```

LISTING 3.6: S-expression representation of the main function preserving its nesting structure

Here, the lambda definitions are nested in direct correspondence with the nesting structure expressed through the `where...end` blocks in Mim. The variable `_2030`, which represents the domain of the main function, is available in the definition of the loop continuation. Similarly, the variable `_2072`, which represents the domain of the loop continuation, is available in the scope containing the definitions of the exit and body continuations.

3.2.3 Encoding Type Information

In the preceding s-expression representations, we have not encoded the types associated with MimIR terms. Transferring this type information to an e-graph serves multiple important functions. Firstly, conditional rewrite rules can perform checks that involve the type of a matched term. Secondly, representing types in an e-graph enables a mechanism of type inference that we explain in more detail in [section 3.4.6](#). Lastly, the reconstruction of an optimized MimIR program through the MimIR API requires type information, such as the types of lambdas.

To achieve this type encoding, we introduced the function symbol `@` to represent type annotations in our s-expressions. An s-expression with this function symbol takes a type in its first position and a term that inhabits the type in its second position. [Listing 3.7](#) illustrates this with a minimal example program in Mim.

```

1  fun extern foo(x: Nat): Nat =
2    return x;

```

LISTING 3.7: Function `foo` defined in Mim

Here, we define the external function `foo` that takes a natural number input parameter `x` and immediately returns it. [Listing 3.8](#) shows this function after it has been converted into the s-expression representation we presented previously.

```

1 (root extern foo
2   (fun _12345 (lit ff)
3     (let return
4       (extract (var _12345) (lit tt))
5     (let x
6       (extract (var _12345) (lit ff))
7     (app (var return) (var x))))))

```

LISTING 3.8: Unannotated s-expression representation of foo

As in previous examples, the s-expression representation of `foo` consists of a root binder that binds the variable `foo` to its function definition. The body of the function again consists of nested `let` bindings for the variables `return` and `x`, where the `return` continuation is finally applied to `x`. Listing 3.9 shows the type-annotated s-expression representation of `foo`.

```

1 (root extern foo
2   (@ (cn (sigma Nat (cn Nat)))
3     (fun _12345 (@ Bool (lit ff))
4       (let return
5         (@ (cn Nat)
6           (extract (@ (sigma Nat (cn Nat)) (var _12345)) (@ Bool (lit tt))))
7       (let x
8         (@ Nat
9           (extract (@ (sigma Nat (cn Nat)) (var _12345)) (@ Bool (lit ff))))
10      (@ bot
11        (app (@ (cn Nat) (var return)) (@ Nat (var x))))))

```

LISTING 3.9: Type-annotated s-expression representation of foo

Every non-binder expression in this representation is surrounded by a type annotation that associates it with its type. For example, the variable use `(var x)` is written as `(@ Nat (var x))`, which means that the type associated with it is `Nat`. The reason why binders are not annotated is that we later infer their types from their bound expressions. This avoids redundant type annotations in the s-expression representation. This s-expression constitutes the final representation of `foo` that we pass on to the equality saturation backends.

3.3 Equality Saturation Plugin

In this section, we present the high-level interface of the equality saturation plugin and the mechanisms for configuring the underlying backends. The plugin, which we named `eqsat`, exposes a set of types and axioms through its `Mim` file. The purpose of these is to provide developers with high-level abstractions in `MimIR` for configuring and interacting with the equality saturation backends. We elaborate on the specific axioms and their use in the following subsection. The `eqsat` plugin also contains a rewrite phase whose primary responsibility is to orchestrate the optimization procedure introduced in section 3.1 and reconstruct an optimized program as `MimIR`. We explain this phase and its implementation in section 3.5.

3.3.1 Plugin Interface

An equality saturation engine typically expects a specification of a cost function and a set of rewrite rules. These are parameters that can vary with every run and are therefore configurable in MimIR. We modeled both with the types and axioms shown in [Listing 3.10](#).

```

1  axm %eqsat.Config:      *;
2  axm %eqsat.Ruleset:    *;
3
4  axm %eqsat.AstSize:     %eqsat.Config;
5  axm %eqsat.AstDepth:   %eqsat.Config;
6
7  axm %eqsat.core:       %eqsat.Ruleset;
8
9  axm %eqsat.rulesets:    {n: Nat} → «n; %eqsat.Ruleset» → %eqsat.Config;
10 axm %eqsat.rules:      {n: Nat, Ts: «n; *»} → «i: n; Ts#i» → %eqsat.Config;
11 axm %eqsat.rules_kind: {n: Nat, Ts: «n; □»} → «i: n; Ts#i» → %eqsat.Config;

```

LISTING 3.10: Configuration types and axioms of the eqsat plugin

Here, the `%eqsat.Config` axiom defines a type, as denoted by its type of `*`, which is inhabited by values specifying configuration options. The cost functions represented by the axioms `%eqsat.AstSize` and `%eqsat.AstDepth` are among the values that inhabit this type. The axiom `%eqsat.Ruleset` defines a type inhabited by collections of rewrite rules that are implemented as part of the equality saturation backends. Individual rulesets are represented by axioms of this type. An example is `%eqsat.core`, which represents a ruleset containing transformations that involve the constructs of the core plugin. In [subsection 3.4.2](#), we explain how a ruleset is defined and integrated with the eqsat plugin.

The `%eqsat.rulesets` axiom is used to select the rulesets applied by the equality saturation engine. Lastly, the `%eqsat.rules` and `%eqsat.rules_kind` axioms provide the ability to specify a set of rewrite rules defined directly in MimIR. Rewrite rules parameterized over types are themselves classified by a kind rather than a type. A limitation in MimIR required us to define distinct axioms for sets of type-level and kind-level rewrite rules. We explain in [subsection 3.4.3](#) how rewrite rules are defined in MimIR and translated into a format compatible with the equality saturation backends.

To provide the ability to switch between equality saturation backends, select only parts of a MimIR program for rewriting, and establish reachability between terms via equality saturation, we defined the group of axioms in [Listing 3.11](#).

```

1  axm %eqsat.egg:         %eqsat.Config;
2  axm %eqsat.slotted:    %eqsat.Config;
3
4  axm %eqsat.select:     {n: Nat, Ts: «n; *»} → «i: n; Ts#i» → %eqsat.Config;
5  axm %eqsat.reaches:   {S: *, E: *} → [s: S, e: E, n: Nat] → %eqsat.Config;

```

LISTING 3.11: Additional axioms of the eqsat plugin

The axioms `%eqsat.egg` and `%eqsat.slotted` are configuration options that can be used to specify an equality saturation backend. The `%eqsat.select` axiom specifies a set of root terms that will be optimized, while the remaining MimIR program is left unchanged. Lastly, we defined the axiom `%eqsat.reaches` to establish whether

a given term can reach another term through equality saturation within a specified iteration limit.

Every phase of a plugin is represented by an axiom to facilitate the specification of a compile pipeline, as we explain in [subsection 3.3.3](#). The rewrite phase of the eqsat plugin is represented by the axioms in [Listing 3.12](#).

```

1  import compile;
2
3  axm %eqsat.eqsat_phase:    %compile.Phase;
4  axm %eqsat.rewrite_egg:    %compile.Phase;
5  axm %eqsat.rewrite_slotted: %compile.Phase;
```

LISTING 3.12: Rewrite phase axioms

The `%eqsat.eqsat_phase` axiom represents a minimal phase that acts as an entry point and dispatches to either the `%eqsat.rewrite_egg` or `%eqsat.rewrite_slotted` phase based on the specified backend configuration.

3.3.2 Configuration

A configuration for the eqsat plugin is composed of the axioms presented in the previous subsection. In MimIR, we represent this configuration with a lambda whose body contains a set of configuration values. We chose this representation as it relies only on existing [IR](#) constructs and facilitates the specification of configurations in Mim and through the MimIR [API](#). The eqsat plugin interprets these values when executing its rewrite phase. We discuss the precise mechanisms of this interpretation in the context of the rewrite phase in [subsection 3.5.1](#).

Mim

[Listing 3.13](#) demonstrates the use of a configuration in a minimal example in Mim.

```

1  plugin core;
2  plugin eqsat;
3
4  fun extern example(a: Nat) =
5    return (%core.nat.add (a, 0));
6
7  lam extern _config() =
8    (
9      %eqsat.egg,
10     %eqsat.AstSize,
11     %eqsat.rulesets (%eqsat.core),
12    );
```

LISTING 3.13: Example eqsat configuration in Mim

Here, we define the external function `example` that adds zero to its natural number input parameter `a`. For this example, we assume that this addition is not normalized by the MimIR compiler and that the ruleset represented by `%eqsat.core` contains the rewrite rule `%core.nat.add (?x, 0) → ?x`.

We specify a configuration for the eqsat plugin via the `_config` lambda, whose body consists of a tuple that contains various configuration values. The `%eqsat.egg` axiom specifies egg as the equality saturation backend, while the `%eqsat.AstSize` axiom selects the [AST](#)-size cost function. Finally, the application `%eqsat.rulesets`

(%eqsat.core) selects the ruleset represented by %eqsat.core for equality saturation. During equality saturation, the rule %core.nat.add (?x, 0) $\rightarrow$?x establishes an equivalence between the return value of the example function %core.nat.add (a, 0) and a. The smaller term a is then extracted according to the **AST**-size cost function.

MimIR API

The eqsat plugin is configured similarly when interacting with it through the MimIR **API**. Since this **API** is overall more verbose than Mim, defining a configuration lambda requires more boilerplate code. To ease this process, we provide the function eqsat_config that abstracts over repetitive calls to factory methods and shifts the focus to the configuration values. This function takes a reference to a World and a set of configuration values and then creates a corresponding lambda in the given World. **Listing 3.14** recreates the same example from **Listing 3.13** through the MimIR **API**.

```

1  auto driver = Driver("example");
2  auto& w     = driver.world();
3  ast::load_plugins(w, View<std::string>{"core", "eqsat"});
4
5  auto nat_t = w.type_nat();
6  auto example = w.mut_fun(nat_t, nat_t)->set("example");
7  auto a       = example->var(2, 0);
8  auto ret     = example->var(2, 1);
9  example->set_filter(false);
10 example->set_body(w.app(ret, w.app(w.annex<core::nat::add>(), a)));
11 example->externalize();
12
13 eqsat_config(
14     w,
15     eqsat::egg,
16     eqsat::AstSize,
17     eqsat_rulesets(eqsat::core),
18 );

```

LISTING 3.14: Example eqsat configuration through the MimIR API

Here, we instantiate a new Driver and create a reference w to its World. Then, the core and eqsat plugins are loaded. We define the example function similarly to our previous example in **Listing 2.9**. Finally, we specify a configuration via the eqsat_config function. We call this function with the values eqsat::egg, eqsat::AstSize, and eqsat_rulesets(eqsat::core), creating a configuration that corresponds directly to the earlier Mim configuration.

3.3.3 Compile Pipelines

During optimization, the MimIR compiler typically applies phases in a fixed order according to what it calls a compile pipeline. For the purposes of testing the eqsat plugin in isolation and to minimize interference with other phases, we needed control over the applied phases. MimIR's compile pipelines can be customized in a similar fashion to the configurations introduced in the previous subsection. A pipeline can be specified through a lambda whose body contains a sequence of phases. In

many of the examples in our work, we relied on a compile pipeline that only executes the rewrite phase of the eqsat plugin. Listing 3.15 illustrates the Mim program from Listing 3.13 extended with a custom compile pipeline.

```

1  plugin compile;
2  plugin core;
3  plugin eqsat;
4
5  fun extern example(a: Nat) =
6    return (%core.nat.add (a, 0));
7
8  lam extern _config() =
9    (
10     %eqsat.egg,
11     %eqsat.AstSize,
12     %eqsat.rulesets (%eqsat.core),
13    );
14
15  lam extern _compile() =
16    %compile.phases ff (
17      %compile.named_phase "%eqsat.eqsat_phase",
18    );

```

LISTING 3.15: Example Mim program extended with a custom compile pipeline

We specify a compile pipeline with the `_compile` lambda. The body of this lambda consists of a curried application of the `%compile.phases` axiom imported with the `compile` plugin. The last argument in this application is a tuple that specifies a phase sequence. In this example, we configured the pipeline to execute only the entry-point phase of the `eqsat` plugin via the application `%compile.named_phase "%eqsat.eqsat_phase"`. The axiom `%compile.named_phase` identifies a phase through a string representation of its corresponding axiom.

3.4 Equality Saturation Backends

To facilitate the optimization of MimIR programs through equality saturation, we implemented two equality saturation backends in `egg` and `slotted-egraphs` and integrated them with the `eqsat` plugin. In this section, we present these implementations in detail, beginning with an overview of the language definition corresponding to the s-expression representation of MimIR. We explain the internal structure of rulesets and how they are integrated into the `eqsat` plugin. We also demonstrate how rewrite rules are defined in MimIR and translated into a format compatible with `egg` and `slotted-egraphs`.

We then describe the process of e-graph construction, saturation, and term extraction, and present our implementation of an e-class analysis responsible for maintaining type information. Finally, we explain our implementation of an FFI to enable interactions between the C++ components of the `eqsat` plugin and the Rust components of the equality saturation libraries. We primarily focus our explanations on the `egg` backend as the two implementations are largely analogous and only differ in a few implementation details.

3.4.1 MimIR Language Definition

To enable the equality saturation backends to interpret the s-expression representation of MimIR that we introduced in [section 3.2](#), we needed to specify the corresponding language variants. We achieved this with the `define_language!` macro presented in [subsection 2.2.4](#). [Listing 3.16](#) illustrates an excerpt of the MimIR language definition in egg.

```

1  define_language! {
2      pub enum Mim {
3          "let"    = Let([Id; 3]),
4          "lam"    = Lam(Box<[Id]>),
5          "con"    = Con(Box<[Id]>),
6          "fun"    = Fun(Box<[Id]>),
7          "app"    = App([Id; 2]),
8          "var"    = Var(Id),
9          "lit"    = Lit([Id; 2]),
10         "pack"    = Pack([Id; 3]),
11         "tuple"   = Tuple(Box<[Id]>),
12         "extract" = Extract([Id; 2]),
13         "insert"  = Insert([Id; 3]),
14         // ...
15         Num(u64), Symbol(String),
16     }
17 }
```

LISTING 3.16: Excerpt of the MimIR language definition in egg

Here, we define the language `Mim` where the function symbols of s-expressions are written on the left-hand sides of their corresponding language variants, as in `"app" = App([Id; 2])`. The `Id` arrays of language variants represent the IDs of their children. Finally, the collection of child `Ids` of variadic constructs, such as `Tuple`, is represented as `Box<[Id]>`.

The language definition of MimIR in slotted-egraphs contained a few minor differences due to the slot-based representation and a limitation of the `define_language!` macro. [Listing 3.17](#) shows an excerpt of the MimIR language definition in slotted-egraphs.

```

1  define_language! {
2      pub enum Mim {
3          Let(Bind<AppliedId>)          = "let",
4          Lam(Bind<AppliedId>)          = "lam",
5          Con(Bind<AppliedId>)          = "con",
6          Fun(Bind<AppliedId>)          = "fun",
7          App(AppliedId, AppliedId)     = "app",
8          Var(Slot)                     = "var",
9          Lit(AppliedId, AppliedId)     = "lit",
10         Pack(Bind<AppliedId>)          = "pack",
11         Tuple(AppliedId)               = "tuple",
12         Extract(AppliedId, AppliedId)  = "extract",
13         Insert(AppliedId, AppliedId, AppliedId) = "insert",
14         // ...
15         Scope(AppliedId, AppliedId)    = "scope",
16         Cons(AppliedId, AppliedId)     = "cons",
17         Nil()                          = "nil",
18         Num(u64),
19         Symbol(Symbol),
20     }
21 }

```

LISTING 3.17: Excerpt of the MimIR language definition in slotted-egraphs

Instead of regular child e-class IDs, slotted-egraphs uses `AppliedIds` that represent a child e-class ID along with a slot mapping. Variants that bind a slot, such as `Lam`, have to be explicitly marked as such with a `Bind` construct. Variants that merely use a slot, in this case `Var`, specify this with the `Slot` construct.

In addition, language variants with a variable number of operands, such as a `Tuple`, were not supported by the `define_language!` macro at the time of our implementation. For this reason, we extended the language definition with binary `Cons` and nullary `Nil` variants to represent the operands of a variadic construct. The operands each occupy the first position of a successive `cons` expression, with the final `cons` expression ending in a single `nil` expression. Listing 3.18 shows a tuple representation in slotted-egraphs whose operands are encoded in a `cons` list.

```

1  (tuple
2      (cons 1
3          (cons 2
4              (cons 3
5                  nil))))

```

LISTING 3.18: A variadic tuple represented with a cons list

Lastly, we introduced the `Scope` variant to facilitate binding a slot in multiple terms instead of only one. This was necessary due to a limitation in the slotted-egraphs library, where a `Bind` construct only binds a slot in a single term. Listing 3.19 illustrates this with a concrete example, where the slot `$foo` is in scope for the two variable uses (`var $foo`) inside the `scope` expression.

```

1 (lam $foo
2   (scope
3     (var $foo)
4     (var $foo)))

```

LISTING 3.19: The slot \$foo in scope for two variable uses

3.4.2 Ruleset Embedding

Rewrite rules defined in MimIR, whose mechanisms we present in more detail in the following subsection, are purely syntactic. A limitation of this mechanism is that these rewrite rules do not capture the full expressiveness provided by the rewrite rule **APIs** of egg and slotted-egraphs, which also include conditional and dynamic rules. In our implementation, we address this limitation by providing developers with the ability to define rewrite rules in MimIR, as well as the ability to define collections of rewrite rules directly in an equality saturation library. We call these collections rulesets, an example of which is represented by the axiom %eqsat.core introduced in [subsection 3.3.1](#). [Listing 3.20](#) shows an excerpt of this ruleset.

```

1 pub fn rules() -> Vec<Rewrite<Mim, MimAnalysis>> {
2   let rules = vec![
3     nat_add0(),
4   ];
5
6   rules
7 }
8
9 fn nat_add0() -> Rewrite<Mim, MimAnalysis> {
10  let pat: Pattern<Mim> = "(app %core.nat.add (tuple ?x (lit 0)))".parse().unwrap();
11  let output: Pattern<Mim> = "?x".parse().unwrap();
12
13  Rewrite::new("nat_add0", pat, output).unwrap()
14 }

```

LISTING 3.20: Excerpt of the ruleset represented by %eqsat.core

Every ruleset lives in its own file whose public interface consists of the `rules()` function that returns a vector of the rewrite rules contained in the ruleset. Each rewrite rule is defined in a function that returns a `Rewrite`, which is egg’s representation of a rewrite rule. In this example, the function `nat_add0()` constructs the rewrite rule `%core.nat.add (?x, 0) → ?x`. It achieves this by parsing the s-expression patterns `(app %core.nat.add (tuple ?x (lit 0)))` and `?x` that represent the respective sides of the rule and then using them to define a new rule via `Rewrite::new()`. This is a purely syntactic rewrite rule; however, the same **API** can also be used to construct more complex conditional or dynamic rewrite rules.

Before constructing an e-graph and attempting to saturate it, the equality saturation backend retrieves the rewrite rules from the rulesets selected through the plugin configuration. To facilitate this process, we defined the interface for interacting with rulesets shown in [Listing 3.21](#).

```

1  thread_local! {
2      pub static RULESETS: RefCell<Vec<RuleSet>> = const { RefCell::new(vec![]) };
3  }
4
5  pub fn set_rulesets(rulesets: Vec<RuleSet>) {
6      RULESETS.with(|rulesets_global| {
7          let mut rulesets_global = rulesets_global.borrow_mut();
8          *rulesets_global = rulesets;
9      });
10 }
11
12 pub mod core;
13 pub fn get_rules() -> Vec<Rewrite<Mim, MimAnalysis>> {
14     let mut rules = Vec::new();
15
16     RULESETS.with(|rulesets_global| {
17         for ruleset in rulesets_global.borrow().iter() {
18             match *ruleset {
19                 RuleSet::Core => rules.extend(core::rules()),
20                 _ => (),
21             }
22         }
23     });
24
25     rules
26 }

```

LISTING 3.21: Interface for ruleset selection and rule retrieval

Here, we defined a thread-local `RULESETS` variable that stores the rulesets selected through a plugin configuration as a vector of `RuleSet` enum variants. We explain how ruleset axioms are mapped to their corresponding `RuleSet` variants in the context of the rewrite phase in [subsection 3.5.1](#). The `set_rulesets()` function acts as a setter for the `RULESETS` variable and populates it with the `rulesets` vector passed to the function. The `get_rules()` function acts as a getter that collects the rewrite rules contained in every ruleset in the `RULESETS` vector and returns them as a combined vector of `Rewrites`. For example, to collect the rules of the `%eqsat.core` ruleset, it matches the corresponding enum variant `RuleSet::Core` and extends the `rules` vector with the return value of the `rules()` function introduced in [Listing 3.20](#).

To ease the process of embedding a new ruleset into the existing code infrastructure, we implemented a Python script that generates all the relevant code, including an axiom, a corresponding `RuleSet` enum variant, a ruleset file, and a match arm in the `get_rules()` function. This enables developers to quickly integrate a new ruleset into the code base and eliminates the need to manually implement the surrounding boilerplate code.

3.4.3 MimIR Rule Translation

For many optimization tasks, a set of purely syntactic rewrite rules is sufficient to express the transformation space. In such cases, the implementation effort involved in the creation of a new ruleset can be avoided by defining the required rewrite rules directly in MimIR. A further benefit of this approach is that rewrite rules are translated at runtime, whereas changes to the backend-defined rulesets require re-compilation. In this subsection, we explain how the equality saturation backends

translate MimIR-defined rewrite rules into their own Rewrite representation. [Listing 3.22](#) shows the definition of a rewrite rule in Mim.

```
1 rule foo(x: Nat):
2   %core.nat.add (x, 0) => x;
```

mim

LISTING 3.22: The rewrite rule `%core.nat.add (?x, 0) → ?x` defined in Mim

Here, we define the rewrite rule `foo` in Mim, which represents the same syntactic rewrite rule that we defined earlier as part of the `%eqsat.core` ruleset in [Listing 3.20](#). For this example, we again assume that the addition `%core.nat.add (x, 0)` is not normalized by the MimIR compiler. The rule takes a natural number input parameter `x`, which represents the pattern variable `?x`. The left- and right-hand sides of the rule are separated by `=>`. It is emitted as an s-expression along with the remainder of the MimIR program. [Listing 3.23](#) illustrates the s-expression representation of `foo`.

```
1 (rule foo
2   (metavar x_22735)
3   (app %core.nat.add (tuple x_22735 (lit 0)))
4   x_22735
5   (lit tt))
```

sexpr

LISTING 3.23: S-expression representation of the rewrite rule `foo`

Here, the rewrite rule is captured in an s-expression with the `rule` function symbol, whose first position is occupied by its name `foo`. The second position is occupied by the pattern variable `x_22735`, which is marked as a pattern variable by its surrounding `metavar` s-expression. As we previously explained, `x` is equipped with a GID suffix in MimIR's internal representation. The third position consists of the left-hand side of the rule, which is an s-expression representation of the pattern `%core.nat.add (?x, 0)`. The fourth position consists of the right-hand side of the rule, representing the pattern `?x`. In the fifth position, the guard of the rule is expressed. The guard consists of an arbitrary Boolean-typed MimIR expression that allows for the definition of conditional rewrite rules. We did not translate this part because we were unable to accurately map its semantics to the conditional rewrite rule mechanisms provided by the equality saturation backends. Therefore, conditional rewrite rules in MimIR do not retain their conditional semantics after translation.

The equality saturation backends translate this s-expression representation into corresponding `Rewrites` through their rewrite rule [API](#). MimIR-defined rewrite rules are translated and collected after the rules contained in rulesets. The rules from the configured rulesets and the MimIR-defined rewrite rules ultimately form a single vector of `Rewrites`. [Listing 3.24](#) shows an excerpt of the code that converts a rule in s-expression format to a `Rewrite` in egg.

```

1  let rule: RecExpr<Mim> = sexpr.parse().unwrap();
2  let [name, _, lhs, rhs, _] = rule[root()].children;
3
4  let name = if let Mim::Symbol(s) = &rule[name] { s } else { "" };
5
6  let mut meta_vars: Vec<String> = Vec::new();
7  lookup(&rule, &mut meta_vars);
8
9  let lhs_rexpr = rule[lhs].build_recexpr();
10 let rhs_rexpr = rule[rhs].build_recexpr();
11 let mut pat:   = lhs_rexpr.pretty();
12 let mut outpat: = rhs_rexpr.pretty();
13
14 inject_meta_vars(&meta_vars, &mut pat);
15 inject_meta_vars(&meta_vars, &mut outpat);
16
17 let rule: Rewrite<Mim, MimAnalysis> = Rewrite::new(name, pat, outpat).unwrap();

```

LISTING 3.24: Excerpt of the translation of a MimIR-defined rewrite rule to egg

Here, the s-expression representation of the rewrite rule is first parsed into a recursive expression `RecExpr<Mim>`. The child indices of its operands are then deconstructed and stored in the variables `name`, `lhs`, and `rhs`. The second and fifth operands, which represent the rule's parameter group and guard, respectively, are discarded. The shadowed name variable is initialized with the name of the rewrite rule, and the names of the rule's pattern variables are identified and stored in the `meta_vars` vector through the `lookup()` function. The left- and right-hand sides are converted back into their s-expression representations via successive calls to `build_recexpr()` and `pretty()` and stored in the respective variables `pat` and `outpat`.

The pattern variables are then injected into these s-expressions through the function `inject_meta_vars()`, which prefixes all occurrences of the pattern variables in `pat` and `outpat` with a question mark. This adjusts emitted s-expression patterns such as `(tuple x_22735 (lit 0))` to the format expected by egg, where this pattern is represented as `(tuple ?x_22735 (lit 0))`. Finally, a new rewrite rule is constructed via `Rewrite::new(name, pat, outpat)`.

3.4.4 Equality Saturation Implementation

In the preceding subsections, we have explained the mechanisms of collecting rules from rulesets and translating MimIR-defined rewrite rules. In this subsection, we demonstrate how the equality saturation backends utilize the combined rule vector to perform equality saturation on an input s-expression representation of a MimIR program. We introduce the `eqsat_egg()` function, which the egg backend exposes on its public interface. This function is responsible for orchestrating the steps of the equality saturation procedure, from the collection of rewrite rules to the optimization of terms. It is exposed through the **FFI** and called from the rewrite phase of the eqsat plugin, as we explain in more detail in [subsection 3.5.1](#). [Listing 3.25](#) shows the implementation of the `eqsat_egg()` function.

```

1  pub fn eqsat_egg(
2      sexpr: &str,
3      rulesets: Vec<RuleSet>,
4      cost_fn: CostFn,
5  ) -> Vec<RecExprFFI> {
6
7      set_rulesets(rulesets);
8      let mut rules = get_rules();
9
10     let mut sexprs = split_sexprs(sexpr);
11     convert_rules(&mut sexprs, &mut rules);
12
13     match cost_fn {
14         CostFn::AstSize => rewrite_sexprs(&sexprs, rules, || AstSize),
15         CostFn::AstDepth => rewrite_sexprs(&sexprs, rules, || AstDepth),
16         _ => panic!("Unknown cost function provided."),
17     }
18 }

```

LISTING 3.25: Public interface for equality saturation in egg

Here, the functions `set_rulesets()` and `get_rules()`, which we introduced with the ruleset interface in [Listing 3.21](#), are used to collect the rewrite rules from the rulesets specified by the input `rulesets` vector. Then, the input s-expression representation of a MimIR program is split into a vector of root-level s-expressions via `split_sexprs()`. We reiterate that the s-expression representation of a MimIR program consists of a series of s-expressions, each of which represents one root term that can be added to an e-graph using egg’s e-graph [API](#). The `convert_rules()` function iterates over this vector and filters out the s-expressions representing MimIR-defined rewrite rules. These are translated as in [Listing 3.24](#) and appended to the `rules` vector.

Lastly, the `rewrite_sexprs()` function is called with the `sexprs`, the `rules`, and a cost function. The input parameter `cost_fn`, which is a variant of the `CostFn` enum, determines the cost function passed to `rewrite_sexprs()`. The `CostFn` enum directly corresponds to a cost function axiom, similarly to the mapping of ruleset axioms to the `RuleSet` enum. We describe these mappings in the context of the rewrite phase in [subsection 3.5.1](#). The `rewrite_sexprs()` function is responsible for initializing an e-graph with the given `sexprs`, attempting to saturate it with the given `rules`, and finally extracting an optimized term for every root in accordance with the provided cost function. In the following, we present isolated excerpts of this function that implement e-graph construction, saturation, and term extraction.

E-Graph Construction

The part of `rewrite_sexprs()` responsible for constructing an e-graph from the input MimIR program is shown in [Listing 3.26](#).

```

1  let mut roots: Vec<Id> = vec![];
2  let mut egraph = EGraph::<Mim, MimAnalysis>::default();
3
4  for sexpr in sexprs {
5      let annotated: RecExpr<Mim> = sexpr.parse().unwrap();
6      let typed:      TypedRecExpr = parse_typed(&annotated);
7
8      let root = add_expr_typed(&mut egraph, typed);
9      roots.push(root);
10 }

```

LISTING 3.26: E-graph construction

Here, we initialize an empty roots vector that will contain the IDs of the e-classes representing the root terms in the e-graph, in the order in which they were added. We also initialize a new EGraph with the default configuration provided by egg and store it in the egraph variable. Then, we iterate over the sexprs representing the MimIR program and add each one to the e-graph. In egg, a parsed s-expression can be added to an EGraph via `EGraph::add_expr()`, but we required a slightly different approach due to the type-annotations contained in our s-expression representation.

We parse a type-annotated s-expression into a regular recursive expression `RecExpr<Mim>` and store it in the annotated variable. This representation is then converted via `parse_typed()` into a corresponding typed recursive expression `TypedRecExpr` and stored in the typed variable. This typed representation separates the annotated portions of the s-expression from their type annotations and equips every expression with a representation of its type. For example, the typed representation of the s-expression `(@ Nat (lit 1))` separates the term `(lit 1)` from its type annotation `Nat` and represents both individually. This separation allows us to add expressions to the e-graph while initializing the analysis data of their e-classes with representations of their types. The `add_expr_typed()` function adds a `TypedRecExpr` to the e-graph analogously to `EGraph::add_expr()` while transferring its type information to the corresponding e-class analysis data. We explain the importance of this in more detail in the context of the type analysis in [subsection 3.4.6](#). Finally, the e-class ID of the newly added expression is stored in `root` and appended to the roots vector.

E-Graph Saturation

[Listing 3.27](#) shows the portion of `rewrite_sexprs()` that attempts to saturate the previously constructed e-graph with the given rules.

```

1  let mut runner = Runner::<Mim, MimAnalysis>::default().with_egraph(egraph);
2
3  let runner = runner.run(&rules);

```

LISTING 3.27: E-graph saturation

Here, we instantiate a new Runner using the default configuration provided by egg and equip it with the egraph representing the input MimIR program. Then, we attempt to saturate this egraph with the previously collected rewrite rules via `runner.run(&rules)`. As we explained in [section 2.2.4](#), the Runner iteratively applies these rewrite rules until the e-graph reaches saturation or a configured limit is reached.

Term Extraction

The part of `rewrite_sexprs()` responsible for extracting the optimized root terms from the e-graph is shown in [Listing 3.28](#).

```

1 let mut rewritten_exprs: Vec<RecExprFFI> = vec![];
2 let extractor = Extractor::new(&runner.egraph, cost_fn());
3
4 for root in roots {
5     let (_, best_expr) = extractor.find_best(root);
6     let best_expr_ffi = best_expr.to_ffi(Some(&runner.egraph));
7
8     rewritten_exprs.push(best_expr_ffi);
9 }

```

LISTING 3.28: Term extraction

Here, we initialize the variable `rewritten_exprs` with an empty vector that will contain an **FFI** representation for each optimized root term. We also instantiate a new `Extractor` with the runner's e-graph and the provided cost function and store it in `extractor`. We iterate over the e-class IDs of the root terms stored in `roots` and extract a lowest-cost representative for each e-class via `extractor.find_best(root)`. This representative is stored in the variable `best_expr` as a `RecExpr<Mim>`. We convert the `best_expr` into its corresponding **FFI** representation `RecExprFFI`, which is then appended to the `rewritten_exprs` vector. This conversion enables the C++ code of the `eqsat` plugin to interact with the representations of the optimized terms. We discuss the structure of a `RecExprFFI` in more detail in [subsection 3.4.7](#).

3.4.5 Reachability Checks

We additionally required a mechanism that allows us to verify whether a given start term becomes equivalent to a given goal term through the process of equality saturation. Typically, following equality saturation, an optimized term is extracted according to a cost function. However, in some cases, we want to establish that a term reaches a specific goal, regardless of whether that goal corresponds to the least-cost equivalent term. In particular, we required this functionality during our evaluation in [chapter 4](#). To achieve this, we implemented the `reaches_egg()` function as part of the public interface of the `egg` backend, whose implementation we present in [Listing 3.29](#).

```

1  pub fn reaches_egg(
2      sexpr: &str,
3      rulesets: Vec<RuleSet>,
4      start_name: &str,
5      goal_name: &str,
6      max_steps: usize,
7  ) -> bool {
8      set_rulesets(rulesets);
9      let mut rules = get_rules();
10
11     let mut sexprs = split_sexprs(sexpr);
12     convert_rules(&mut sexprs, &mut rules);
13
14     let start = get_term(&sexprs, start_name);
15     let goal = get_term(&sexprs, goal_name);
16
17     check_reaches(&start, &goal, &rules, max_steps)
18 }

```

LISTING 3.29: Public interface for reachability checks in egg

As in the previous `eqsat_egg()` function, the `reaches_egg()` function collects the rewrite rules and splits the input into individual s-expressions. Then, it extracts the s-expression corresponding to the `start_name` parameter through `get_term(&sexprs, start_name)` and stores it in `start`. For example, the s-expression representation of a function `foo` is identified by its name `foo`. Similarly, the s-expression corresponding to `goal_name` is retrieved and stored in `goal`. Finally, the `check_reaches()` function checks whether the `start` term reaches the `goal` term within an equality saturation iteration limit of `max_steps`. We present an excerpt of its implementation in [Listing 3.30](#).

```

1  let start: RecExpr<Mim> = start.parse().unwrap();
2  let goal: RecExpr<Mim> = goal.parse().unwrap();
3
4  let runner = Runner<Mim, MimAnalysis>::default()
5      .with_expr(&start)
6      .with_iter_limit(max_steps)
7      .with_hook(reach_hook(&start, &goal, max_steps));
8  let runner = runner.run(rules);
9  let report = runner.report();
10
11 matches!(report.stop_reason, StopReason::Other(reason) if reason == "Reached")

```

LISTING 3.30: Equality saturation for reachability checking in `check_reaches()`

The `check_reaches()` function begins by parsing the provided `start` and `goal` s-expressions as recursive expressions. It instantiates a new `Runner` and initializes its e-graph with the `start` term. This `Runner` is also equipped with an iteration limit of `max_steps` and a hook `reach_hook()`, which is responsible for checking whether `start` and `goal` are equivalent after every iteration. Then, equality saturation is performed via `runner.run(rules)`, and a report of this run is stored in `report`. Finally, the `check_reaches()` function returns true if the termination reason generated by `reach_hook()` indicates that the `start` term reached the `goal` term. We present the relevant excerpt of `reach_hook()` in [Listing 3.31](#).

```

1 if !runner.egraph.equivs(&start, &goal).is_empty() {
2     return Err("Reached".to_owned());
3 }
4 Ok(())

```

LISTING 3.31: Reachability check in `reach_hook()`

The `reach_hook()` is executed after every iteration of equality saturation and halts this process if the provided `start` and `goal` terms are represented by the same e-class. In this case, the reachability check is successful and equality saturation is terminated with the reason "Reached".

3.4.6 Type Analysis

To maintain the typing relation during equality saturation and associate newly added terms with their corresponding types, we implemented an e-class analysis that stores type information as analysis data. One significant benefit of this approach is that it allows for the definition of conditional rewrite rules that inspect the types of matched terms. Another benefit is that optimized terms are equipped with type information, which is required to reconstruct them as MimIR terms because certain expressions, such as lambdas, require a type upon construction.

The `make` and `merge` functions of an e-class analysis naturally correspond to the mechanisms of type inference and unification. The `make` function is executed after a new e-node has been added to the e-graph and derives analysis data from it. In the context of our type analysis, deriving a new type from an e-node corresponds to inferring the type of a term. The `merge` function defines a join operation on the analysis data of two e-classes when they are judged equivalent and combined. In the context of our type analysis, this corresponds to the unification of the types associated with the two groups of equivalent terms. In the following, we describe the high-level interface of the type analysis and explain the implementations of the `make` and `merge` functions.

Analysis Interface

To represent type expressions as analysis data, we chose to model them with egg's `RecExpr<Mim>` format. We defined `TypeExpr` as a type alias for `RecExpr<Mim>` and further implemented constructors for the different type variants. This allowed us to construct, for example, a new `Sigma` using `TypeExpr::sigma()`. The `AnalysisData` struct represents the data associated with every e-class. We defined the `type_` field of type `Option<TypeExpr>` on `AnalysisData` to optionally associate an e-class with a type. E-classes containing structural e-nodes, such as `Cons` and `Nil`, are not associated with a type because they have no corresponding terms in MimIR. Lastly, the analysis is represented by the `TypeAnalysis` struct, which implements the `make()` and `merge()` functions. Listing 3.32 shows the high-level interface of our type analysis.

```

1  pub type TypeExpr = RecExpr<Mim>;
2
3  pub struct AnalysisData {
4      type_: Option<TypeExpr>,
5  }
6
7  pub struct TypeAnalysis;
8
9  impl TypeAnalysis {
10     pub fn make(eg: &EGraph<Mim, MimAnalysis>, enode: &Mim, _id: Id) -> AnalysisData {
11         // ...
12     }
13     pub fn merge(l: &mut AnalysisData, r: AnalysisData) -> DidMerge {
14         // ...
15     }
16 }

```

LISTING 3.32: Type analysis interface

Type Inference

The `make()` function of the type analysis performs type inference for newly added terms. To demonstrate this process with a concrete example, we define the Mim program in [Listing 3.33](#).

```

1  axm %foo.bar: [Nat, Bool];
2
3  rule baz(): %foo.bar => (1, tt);
4
5  fun extern example() =
6      return %foo.bar;

```

LISTING 3.33: Example Mim program for type inference

In this example, we declare the opaque axiom `%foo.bar` of type `[Nat, Bool]`. We also define the rewrite rule `baz`, which transforms this axiom into the concrete tuple `(1, tt)`. During equality saturation, `baz` matches the term `%foo.bar` in the body of the `example` function, and the e-graph is extended with a representation of the tuple `(1, tt)`. First, the e-nodes representing the tuple elements `1` and `tt` are added, followed by the e-node representing the tuple. The `make()` function infers the `Nat` and `Bool` types for the e-classes representing the literals `1` and `tt`. Finally, the type of the tuple e-class is inferred as a `Sigma` containing the element types `Nat` and `Bool`. To explain how the `make()` function performs the type inference demonstrated in this example, we present its implementation in [Listing 3.34](#).

```

1 fn make(eg: &EGraph<Mim, MimAnalysis>, enode: &Mim) -> AnalysisData {
2     match enode {
3         Mim::Let(..) => make_let_type(eg, enode),
4         Mim::Lam(..) => make_lam_type(eg, enode),
5         Mim::Con(..) => make_con_type(eg, enode),
6         Mim::Fun(..) => make_fun_type(eg, enode),
7         Mim::App(..) => make_app_type(eg, enode),
8         Mim::Var(..) => make_var_type(eg, enode),
9         Mim::Lit(..) => make_lit_type(eg, enode),
10        Mim::Pack(..) => make_pack_type(eg, enode),
11        Mim::Tuple(..) => make_tuple_type(eg, enode),
12        Mim::Extract(..) => make_extract_type(eg, enode),
13        Mim::Insert(..) => make_insert_type(eg, enode),
14        // ...
15
16        _ => AnalysisData {
17            type_: Some(TypeExpr::hole()),
18            ..Default::default()
19        },
20    }
21 }

```

LISTING 3.34: Type inference dispatch in make()

Recall that the `make()` function is called with every addition of a new e-node to the e-graph. Its input parameters consist of the e-graph `eg` and the newly added `enode`. It returns the analysis data derived from the `enode`. The `make()` function matches the variant of the `enode` to delegate to the corresponding type inference implementation. In the previous example, this would have resulted in calls to `make_lit_type()` and `make_tuple_type()`. Variants not explicitly covered by this match are associated with a type hole. A hole acts as a placeholder for an unknown type, which the MimIR compiler can often infer from context. Listing 3.35 shows the implementation of type inference for Tuple e-nodes in `make_tuple_type()`.

```

1 fn make_tuple_type(eg: &EGraph<Mim, MimAnalysis>, enode: &Mim) -> AnalysisData {
2     let child_ids = expect!(enode, Mim::Tuple(child_ids) => child_ids);
3
4     let mut types: Vec<TypeExpr> = Vec::new();
5
6     for child in child_ids.iter() {
7         let type_ = eg[*child].data.type_.clone();
8         types.push(type_.unwrap_or(TypeExpr::hole()));
9     }
10
11    AnalysisData {
12        type_: Some(TypeExpr::sigma(types)),
13        ..Default::default()
14    }
15 }

```

LISTING 3.35: Tuple type inference

Here, the child e-class IDs of the tuple are extracted with the `expect!` macro and stored in the `child_ids` variable. We iterate over the child e-class IDs and collect the type associated with each corresponding e-class in the `types` vector. If a child e-class does not have an associated type, a type hole is used instead. Finally, we construct

a new `Sigma` from the collected types and return it as part of a new `AnalysisData` struct.

Type Unification

The `merge()` function of the type analysis unifies the types of e-classes being merged. Type unification in this context combines the types of two e-classes such that the resulting type contains no more holes than either original type. The purpose of this is to preserve or increase the available type information during merges. To demonstrate this with a concrete example, we define the Mim program in [Listing 3.36](#).

```
1 axm %foo.bar: _ → Nat;
2 axm %foo.baz: Nat → _;
3
4 rule baz(): %foo.bar => %foo.baz;
5
6 fun extern example() =
7   return %foo.bar;
```

LISTING 3.36: Example Mim program for type unification

In this example, we define two opaque axioms `%foo.bar` and `%foo.baz` whose function types contain holes denoted by `_`. We also define a rewrite rule `baz` that transforms `%foo.bar` into `%foo.baz`. During equality saturation, this rewrite rule matches the term `%foo.bar` in the body of the `example` function. This causes the e-classes containing the representations of `%foo.bar` and `%foo.baz` to be merged. As a result, the incomplete function types `_ → Nat` and `Nat → _` associated with their e-classes are unified into the complete type `Nat → Nat`. [Listing 3.37](#) shows how the type unification demonstrated in this example is implemented in the `merge()` function.

```
1 fn merge(l: &mut AnalysisData, r: AnalysisData) -> DidMerge {
2   match (&l.type_, r.type_) {
3     (None, None) => DidMerge(false, false),
4     (Some(_), None) => DidMerge(false, true),
5     (None, Some(t)) => { l.type_ = Some(t); DidMerge(true, false) }
6     (Some(t1), Some(t2)) => {
7       let unified = unify(t1, &t2);
8
9       let l_changed = *t1 != unified;
10      let r_changed = t2 != unified;
11
12      if l_changed {
13        l.type_ = Some(unified);
14      }
15
16      DidMerge(l_changed, r_changed)
17    }
18  }
19 }
```

LISTING 3.37: Type unification via `merge()`

The `merge()` function is called when two e-classes are combined because they were established as equivalent by a rewrite rule or the congruence relation. We pattern-match the optional types associated with the two e-classes. When neither

e-class is associated with a type, the resulting e-class also remains without an associated type. When only one of the two e-classes is associated with a type, the existing type is propagated according to the mechanism described in [section 2.2.4](#). When both e-classes are associated with a type, we dispatch to the type unification function `unify()` and store the result in `unified`. If the original type of the left e-class differs from the unified type, we update its analysis data with the unified type. This is in accordance with egg’s mechanism for propagating merged analysis data. [Listing 3.38](#) shows the implementation of the `unify()` function.

```

1  fn unify(l: &TypeExpr, r: &TypeExpr) -> TypeExpr {
2      match (&l[l.root()], &r[r.root()]) {
3
4          (_, Mim::Hole(_)) => l.clone(),
5          (Mim::Hole(_), _) => r.clone(),
6
7          (Mim::Pi(_), Mim::Pi(_)) => {
8              let l_dom = child!(l, 1);
9              let l_codom = child!(l, 2);
10
11              let r_dom = child!(r, 1);
12              let r_codom = child!(r, 2);
13
14              let dom = unify(&l_dom, &r_dom);
15              let codom = unify(&l_codom, &r_codom);
16
17              TypeExpr::pi(dom, codom)
18          }
19          // ...
20
21          _ => l.clone(),
22      }
23 }

```

LISTING 3.38: Type unification in `unify()`

The `unify()` function takes references to two type expressions `l` and `r`. Structured types are unified recursively by unifying their corresponding sub-expressions. When `unify()` is called with a type hole and another type, as expressed in the first two match arms, the other type is propagated. The unification of two function types, represented by `Pi` types in `MimIR`, is implemented in the third match arm. Here, the domain and codomain of the left `Pi` type are extracted with the `child!` macro and stored in `l_dom` and `l_codom`. Similarly, the domain and codomain of the right `Pi` type are extracted and stored in `r_dom` and `r_codom`. The domains of the left and right `Pi` types are unified through the recursive call `unify(&l_dom, &r_dom)`, and the result is stored in `dom`. The codomains are unified in the same manner. Finally, a new `Pi` type is constructed via `TypeExpr::pi(dom, codom)`. In the previous example, this recursive process resulted in the unification of the incomplete `Pi` types `_ → Nat` and `Nat → _` into the complete type `Nat → Nat`. Type combinations not explicitly covered by the unification procedure retain the left type.

Limitations

One significant limitation in our approach concerns the maintenance of types whose validity depends on a surrounding typing context. For example, a dependent pack whose arity is defined by a variable, as in the body of the lambda `lam(a: Nat) =`

$\langle a; 2 \rangle$, is associated with the array type $\langle a; \text{Nat} \rangle$. This type is stored in isolation on the e-class representing the term $\langle a; 2 \rangle$ despite depending on the variable a bound by the lambda. During equality saturation, the surrounding binder structure may be altered such that the variable a is no longer bound in the optimized term. Because the type analysis does not maintain the corresponding typing context, this change in binder structure is not reflected in the stored type $\langle a; \text{Nat} \rangle$. As a result, our type analysis cannot guarantee the validity of types that depend on variables bound in a surrounding context.

3.4.7 Foreign Function Interface

To enable the C++ components of the eqsat plugin to interact with the Rust components of the equality saturation backends, we were required to implement an FFI. This allows the rewrite phase to call the `eqsat_egg()` function we introduced in Listing 3.25, pass it a configuration derived from the ruleset and cost function axioms, and interact with optimized terms in the `RecExprFFI` format defined on this interface. We implemented the FFI with the Rust library CXX, which generates corresponding C++ definitions for types and functions declared in a shared Rust interface. Listing 3.39 shows an excerpt of the FFI definitions for the `RuleSet` and `CostFn` enums.

```

1  #[cxx::bridge]
2  pub mod bridge {
3      enum RuleSet {
4          Core,
5      }
6
7      enum CostFn {
8          AstSize,
9          AstDepth,
10     }
11     // ...
12 }
```

LISTING 3.39: FFI definitions for configuration enums in Rust

Here, the `bridge` module encapsulates the declarations for which CXX generates corresponding C++ definitions. We define the `RuleSet` and `CostFn` enums, which serve as shared representations of the corresponding configuration axioms. For example, the axiom `%eqsat.AstSize` is represented as the enum variant `CostFn::AstSize` before being passed to the equality saturation backend. Listing 3.40 shows the C++ definitions generated by CXX for these enums.

```

1  enum class RuleSet : ::std::uint8_t {
2      Core = 0,
3  };
4
5  enum class CostFn : ::std::uint8_t {
6      AstSize = 0,
7      AstDepth = 1,
8  };
```

LISTING 3.40: Generated C++ definitions for configuration enums

The enum classes generated by CXX correspond to the Rust enums and use `std::uint8_t` as their underlying type. Due to limitations in CXX, we were unable to directly expose `RecExpr<Mim>` through the bridge and therefore defined our own representation `RecExprFFI`. In addition to modeling the structure of a `RecExpr<Mim>`, this representation associates each sub-expression with the type extracted from the analysis data of its e-class. [Listing 3.41](#) shows the remaining relevant declarations of the FFI, including the definition of `RecExprFFI` and the declarations of functions such as `eqsat_egg()` and `reaches_egg()`.

```

1  #[cxx::bridge] Rust
2  pub mod bridge {
3      // ...
4      enum MimKind {
5          Let,
6          Lam,
7          Con,
8          Fun,
9          App,
10         // ...
11     }
12
13     struct NodeFFI {
14         kind: MimKind,
15         children: Vec<u32>,
16         num: u64,
17         symbol: String,
18         slot: String,
19         type_: RecExprFFI,
20     }
21
22     struct RecExprFFI {
23         nodes: Vec<NodeFFI>,
24     }
25
26     extern "Rust" {
27         fn eqsat_egg(...);
28         fn reaches_egg(...);
29         // ...
30     }
31 }
```

LISTING 3.41: FFI representation of recursive expressions in Rust

A `RecExprFFI` consists of a vector of language nodes `nodes` of type `NodeFFI`. Similarly to the nodes in a `RecExpr<Mim>`, each `NodeFFI` is classified by a variant of the `MimKind` enum. Child nodes are referenced through the `children` vector, whose elements are indices into the `nodes` vector. Variant-specific data is represented by the `num`, `symbol`, and `slot` fields. The `type_` field itself contains another `RecExprFFI`, converted from the `TypeExpr` stored as analysis data on the corresponding e-class. The `extern "Rust"` block exposes the public interface of the equality saturation backends, which includes the `eqsat_egg()` and `reaches_egg()` functions. For brevity, we omit the corresponding C++ definitions generated by CXX.

3.5 Reconstructing MimIR

Following the optimization of a MimIR program, the eqsat plugin reconstructs the optimized terms represented in RecExprFFI format as corresponding MimIR terms. The rewrite phase is responsible for orchestrating the equality saturation procedure and reconstructing the optimized terms in MimIR. In this section, we provide an overview of the rewrite phase interface and its role in the equality saturation procedure. We then explain the reconstruction of optimized terms, which is performed in two traversals that we present separately. Finally, we elaborate on the reconstruction of the types associated with the optimized terms.

3.5.1 Rewrite Phase

The rewrite phase coordinates the individual steps of the equality saturation procedure and reconstructs the resulting terms in MimIR.

Interface

We present the interface of the rewrite phase in [Listing 3.42](#).

```

1  class RewriteEgg : public Phase, public Rewriter {
2  public:
3      // ...
4      void start() override;
5      World& old_world() { return Phase::world(); }
6      World& new_world() { return Rewriter::world(); }
7
8  private:
9      // ...
10     typedef struct RecExprState {
11         Cache cache;
12         Nodes nodes;
13     } RecExprState;
14
15     typedef struct Context {
16         size_t id;
17         RecExprState* state;
18     } Context;
19
20     Sym2Def vars_;
21     Sym2Def axms_;
22     Sym2Def aliases_;
23     Context ctx_;
24     RecExprStates states_;
25 };

```

LISTING 3.42: Rewrite phase interface

The RewriteEgg phase is responsible for optimizing MimIR programs with the egg backend. It inherits from the Phase and Rewriter base classes, providing it with the same base implementation introduced in [subsection 2.1.5](#). The start() method marks its entry point and is called by the MimIR compiler when the phase is executed. The two getters old_world() and new_world() provide access to the program before and after it has been altered through the phase, respectively.

The `RecExprState` and `Context` types maintain the state required while reconstructing a `RecExprFFI`. Each `RecExprState` stores a copy of the nodes of a `RecExprFFI` and a hash map cache that maps node indices to their corresponding reconstructed MimIR terms. The purpose of the cache is to avoid reconstructing terms that have already been created in MimIR. The `Context` identifies the `RecExprFFI` currently being reconstructed. Its `id` field stores the corresponding index in the vector of `RecExprFFIs` returned by the equality saturation backend, while its `state` pointer references the associated `RecExprState`. The `states_` member maps these indices to their associated `RecExprState` instances. The `ctx_` member stores the context of the `RecExprFFI` currently being reconstructed.

The `vars_`, `axms_`, and `aliases_` members are hash maps that associate symbols with their corresponding MimIR terms. The `vars_` map stores variable symbols, for example associating `foo_12345` with its corresponding MimIR definition. Similarly, `axms_` maps axiom symbols such as `%core.nat.add` to their MimIR definitions. Finally, `aliases_` maps alias symbols such as `tt` to their corresponding MimIR definitions.

Execution

Listing 3.43 shows an excerpt from the implementation of the `start()` function, which illustrates how it orchestrates the equality saturation procedure.

```

1  void RewriteEgg::start() {
2      auto [rulesets, cost_fn, reach_tuples] = import_config();
3
4      std::ostringstream sexpr;
5      sexpr::emit_typed(old_world(), sexpr);
6
7      check_reachability(sexpr.str(), rulesets, reach_tuples);
8
9      auto rec_exprs = eqsat_egg(sexpr.str(), rulesets, cost_fn);
10
11     init(rec_exprs);
12     convert(rec_exprs);
13 }
```

LISTING 3.43: Equality saturation orchestration in `start()`

Here, the configuration values are parsed via `import_config()` and stored in the variables `rulesets`, `cost_fn`, and `reach_tuples`. We explain the implementation of the `import_config()` function in more detail below. Then, the s-expression emitter is invoked with a call to `sexpr::emit_typed()`, which stores an s-expression representation of the MimIR program from the `old_world()` in the variable `sexpr`. The `check_reachability()` function then performs the reachability checks specified by the tuples in `reach_tuples`. The `sexpr`, together with the `rulesets` and `cost_fn` configuration values, is then passed to the `eqsat_egg()` function, which returns the optimized terms as a vector of `RecExprFFIs` stored in `rec_exprs`. Finally, the optimized terms are reconstructed as MimIR terms through successive calls to `init(rec_exprs)` and `convert(rec_exprs)`. We present the traversals implemented by these functions in the following subsections.

Configuration Parsing

Listing 3.44 shows an excerpt from the `import_config()` function, which parses a user-defined configuration of the eqsat plugin.

```

1  CostFn cost_fn = CostFn::AstSize;
2
3  for (auto def : world().externals()) {
4      if (auto lam = def->isa<Lam>()) {
5          if (auto arr = lam->codom()->isa<Arr>()) {
6              arr && Axi::isa<eqsat::Config>(arr->body()) {
7                  auto body      = lam->as<Lam>()->body();
8                  auto configs   = body->as<Tuple>()->ops();
9                  for (auto config : configs)
10                     if (Axi::isa<eqsat::AstSize>(config))
11                         cost_fn = CostFn::AstSize;
12                     // ...
13             }
14         }
15     }

```

LISTING 3.44: Configuration parsing in `import_config()`

Recall that a plugin configuration is specified through a lambda whose body consists of a tuple of configuration axioms. The `import_config()` function identifies such lambdas by iterating over external terms via `world().externals()`. A runtime type check via `def->isa<Lam>()` determines whether an external term is a lambda, while lambdas whose codomain is an array of `%eqsat.Config` axioms specify a configuration for the eqsat plugin. The configuration axioms contained in the tuple are stored in `configs` and iterated over. Specific configuration axioms such as `%eqsat.AstSize` are then identified through a runtime type check via `Axi::isa<eqsat::AstSize>()` and mapped to the corresponding enum variant `CostFn::AstSize`. Each configuration parameter is initialized with a default value, which serves as a fallback when no corresponding configuration axiom is specified. In the case of multiple configuration lambdas, each parameter assumes the last configuration value specified for it.

3.5.2 Initialization

The `init()` function performs an initialization traversal over each recursive expression to reconstruct every binder sub-expression, such as let bindings or lambdas. The purpose of this traversal is to ensure that variables are associated with valid MimIR definitions before terms that reference them are reconstructed. We demonstrate this traversal on a concrete example shown in **Listing 3.45**.

```

1  fun extern foo(x: Nat) =
2      return x;

```

LISTING 3.45: Example Mim function for the initialization traversal

Here, we define the minimal function `foo` that takes a natural number parameter `x` and returns it. The unannotated s-expression representation of `foo` is shown in **Listing 3.46**.

```

1 (root extern foo
2   (fun _123 (lit ff)
3     (let return
4       (extract _123 (lit tt))
5     (let x
6       (extract _123 (lit ff))
7     (app return x))))))

```

LISTING 3.46: S-expression representation of the initialization example

As in previous examples, `foo` is bound to its function definition through a root binder. The variables `return` and `x` are bound via nested `let` bindings in the body of `fun`, while `(app return x)` represents the application `return x`. Figure 3.3 illustrates a graphical representation of the recursive expression derived from this s-expression.

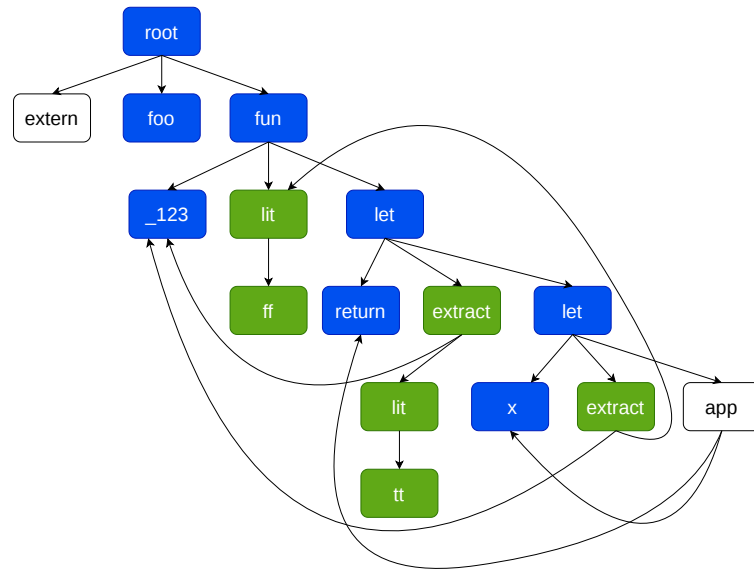


FIGURE 3.3: DAG representation of the initialization example

The structure of a recursive expression can be viewed as a **DAG** with a single root node. The initialization traversal descends from the root of this **DAG** and reconstructs the binder sub-expressions highlighted in blue. The sub-expressions highlighted in green are reconstructed through isolated bottom-up traversals implemented by the `convert()` function. This function is responsible for reconstructing non-binder expressions, as we explain in the following subsection. These partial reconstructions create the MimIR definitions to which the variables introduced by the `let` bindings are bound. After initialization, the variables `foo`, `_123`, `return`, and `x` are associated with their MimIR definitions in the `vars_` hash map. To explain how the initialization demonstrated in this example is performed, we present the implementation of the `init()` function in Listing 3.47.

```

1  void RewriteEgg::init(rust::Vec<RecExprFFI> rec_exprs) {
2      for (size_t i = 0; i < rec_exprs.size(); i++) {
3          auto rec_expr = rec_exprs[i];
4          create_state(i, rec_expr);
5          switch_context(i);
6
7          auto root_id = root();
8          init(root_id);
9      }
10 }

```

LISTING 3.47: Iteration over recursive expressions during initialization

The initialization procedure is implemented by two overloaded `init()` functions that are responsible for iterating over the recursive expressions and traversing them, respectively. Here, we show the implementation of the iteration function, which takes a vector of recursive expressions `rec_exprs` as a parameter. It iterates over the indices of `rec_exprs` and stores the recursive expression at the current index in `rec_expr`. Then, it instantiates a `RecExprState` for the recursive expression via `create_state(i, rec_expr)`. The context is switched to the current recursive expression via `switch_context(i)`. Finally, the traversal function `init()` is called with the ID of the current recursive expression's root node `root_id`. Listing 3.48 shows its implementation.

```

1  const Def* RewriteEgg::init(uint32_t id) {
2      auto node = get_node(id);
3
4      const Def* res = cache()[id];
5      if (!res) {
6          switch (node.kind) {
7              case MimKind::Root: res = init_root(id, node); break;
8              case MimKind::Fun:
9              case MimKind::Con:
10             case MimKind::Lam: res = init_lam(id, node); break;
11             case MimKind::Let: res = init_let(id, node); break;
12             // ...
13             default: break;
14         }
15     }
16
17     for (uint32_t child : node.children)
18         init(child);
19
20     return cache()[id] = res;
21 }

```

LISTING 3.48: Initialization traversal of a recursive expression

The traversal function takes an `id` parameter that represents the ID of the node being visited in the current recursive expression. The corresponding node of the recursive expression is retrieved via `get_node(id)`. Then, the cached MimIR definition at index `id` is retrieved and stored in `res` to determine whether the node has already been reconstructed. If this is not the case, the initialization function for the specific node variant is called to create its MimIR definition. After initializing the

current node, the `init()` function is recursively called on its child IDs, which results in a top-down traversal of the recursive expression. Finally, the result of the initialization is stored in the cache and returned. [Listing 3.49](#) shows the implementation of the `init_lam()` function, which initializes lambda terms.

```

1  const Def* RewriteEgg::init_lam(uint32_t id, NodeFFI node) {
2      auto pi_type = create_type(node.type_)->as<Pi>();
3      auto lam     = new_world().mut_lam(pi_type);
4
5      auto var     = lam->var();
6      auto var_name = get_symbol(node.children[0]);
7      register_var(var_name, var);
8
9      return lam;
10 }
```

LISTING 3.49: Lambda initialization in `init_lam()`

The `init_lam()` function takes the node representing a lambda and its node ID `id` as parameters. It reconstructs the function type of the lambda via `create_type(node.type_)` and stores it in `pi_type`. We explain the implementation of `create_type()` in [subsection 3.5.4](#). Then, it constructs a new mutable lambda without a body via `new_world().mut_lam(pi_type)`. The MimIR definition of the lambda's variable is retrieved via `lam->var()` and its name is retrieved from the first child of the lambda node. Finally, the `var_name` is associated with its MimIR definition `var` in the `vars_` hash map through `register_var()`, and the newly created lambda is returned.

3.5.3 Conversion

Following initialization, the `convert()` function performs a bottom-up traversal of each recursive expression to reconstruct the MimIR definitions that were not created during initialization and complete the initialized lambdas by reconstructing their bodies. The bottom-up traversal ensures that the MimIR definitions corresponding to a node's children are available before reconstructing the node itself. The `convert()` function can also be invoked to reconstruct individual sub-expressions, as during initialization, where it reconstructs the definitions of let-bound variables. [Figure 3.4](#) illustrates the recursive expression shown in [Figure 3.3](#) with the additional sub-expressions reconstructed during conversion.

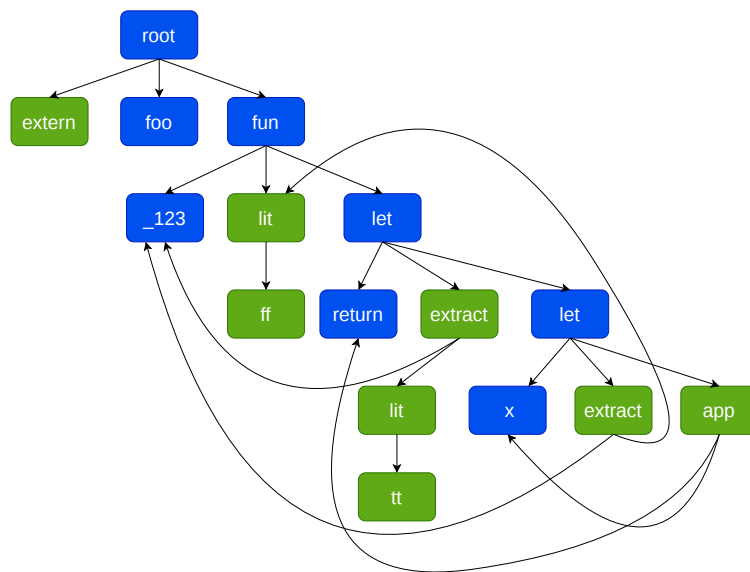


FIGURE 3.4: DAG representation of the conversion example

Here, the function body of `foo`, which consists of the application `return x`, has been reconstructed. Additionally, the function has been externalized as specified by its root binder. We explain how this conversion is performed by presenting the implementation of the `convert()` function in [Listing 3.50](#).

```

1 void RewriteEgg::convert(rust::Vec<RecExprFFI> rec_exprs) {
2     for (size_t i = 0; i < rec_exprs.size(); i++) {
3         auto rec_expr = rec_exprs[i];
4         switch_context(i);
5     }
6     auto root_id = root();
7     convert(root_id);
8 }
9 }
```

LISTING 3.50: Iteration over recursive expressions during conversion

As in the initialization procedure, the conversion is implemented in two overloaded `convert()` functions responsible for iterating over the recursive expressions and traversing them, respectively. The implementation of the iteration function is analogous to the corresponding `init()` function, with the main difference that it does not instantiate a new `RecExprState`, as this has already been done during initialization. The traversal function `convert()` is shown in [Listing 3.51](#).

```

1  const Def* RewriteEgg::convert(uint32_t id) {
2      auto node = get_node(id);
3
4      for (uint32_t child : node.children)
5          convert(child);
6
7      const Def* res = cache()[id];
8      if (res) return res;
9
10     switch (node.kind) {
11         case MimKind::Root: res = convert_root(id, node); break;
12         case MimKind::Let: res = convert_let(id, node); break;
13         case MimKind::Fun:
14         case MimKind::Con:
15         case MimKind::Lam: res = convert_lam(id, node); break;
16         case MimKind::App: res = convert_app(id, node); break;
17         // ...
18         default: break;
19     }
20
21     return cache()[id] = res;
22 }

```

LISTING 3.51: Convert traversal of a recursive expression

This implementation also closely resembles the corresponding `init()` traversal function, with the main difference that child nodes are recursively visited before the current node is reconstructed in MimIR, resulting in a bottom-up traversal of the recursive expression. Listing 3.52 shows the implementation of the `convert_app()` function responsible for reconstructing applications.

```

1  const Def* RewriteEgg::convert_app(uint32_t id, NodeFFI node) {
2      auto callee = get_def(node.children[0]);
3      auto arg = get_def(node.children[1]);
4      auto new_app = new_world().app(callee, arg);
5      return new_app;
6  }

```

LISTING 3.52: Application reconstruction in `convert_app()`

The `convert_app()` function takes the application node and its node ID `id` as parameters. It retrieves the MimIR definitions associated with the node's children, which represent the application's callee and argument, and stores these definitions in `callee` and `arg`. Finally, it creates a new application through `new_world().app(callee, arg)` and returns it.

3.5.4 Type Reconstruction

The types associated with the nodes of a `RecExprFFI` are also represented in the `RecExprFFI` format. This allowed us to reconstruct them with the same initialization and conversion traversals described above. In the `init_lam()` function in Listing 3.49, we used the `create_type()` function to reconstruct the function type of a lambda. Listing 3.53 shows the implementation of this function.

```
1  const Def* RewriteEgg::create_type(RecExprFFI type_) {  
2      auto outer_ctx = ctx();  
3  
4      create_state(SIZE_MAX, type_);  
5      switch_context(SIZE_MAX);  
6  
7      auto type_root = root();  
8  
9      init(type_root);  
10     auto res = convert(type_root);  
11  
12     switch_context(outer_ctx);  
13     return res;  
14 }
```

LISTING 3.53: Type reconstruction in `create_type()`

Here, we store the context of the current recursive expression in the `outer_ctx` variable. This allows us to create an isolated context for type reconstruction and later switch back to the context of the surrounding recursive expression. We instantiate a new `RecExprState` for the type and associate it with the index `SIZE_MAX` to avoid clashing with existing indices. Then, we switch to this context via `switch_context(SIZE_MAX)` and store the ID of the type expression's root node in `type_root`. The type is reconstructed through successive calls to `init(type_root)` and `convert(type_root)`. The resulting MimIR definition is stored in `res` and returned after the context is switched back to `outer_ctx`.

3.5.5 Limitations

We presented the reconstruction of MimIR terms through a process that involves successive initialization and conversion traversals. As demonstrated in the initialization of `let` bindings, sub-expressions are partially reconstructed through isolated conversion traversals during initialization. The same isolated conversion traversals could have been used to reconstruct the entire program during initialization, making the second conversion traversal of each recursive expression unnecessary. This consolidation into a single traversal may lead to a faster reconstruction procedure. However, as the following section shows, reconstruction was relatively fast for all programs we evaluated and did not become a performance bottleneck.

Chapter 4

Case Study: RISE

To evaluate the effectiveness of the equality saturation plugin in a real application, we conducted a case study on the functional RISE programming language developed by Hagedorn et al. (2020). RISE uses data-parallel patterns for computations over multi-dimensional arrays and was intentionally modeled as a functional, pattern-based language to enable optimization through equational rewriting and to generate high-performance code. This made it a particularly good fit for our work, as we are able to demonstrate how the RISE language can be modeled in MimIR and how such programs can be optimized through our equality saturation plugin. We relied on the existing RISE literature to design a case study with the aim of reproducing previously demonstrated program transformations and contrasting the performance of the eqsat plugin with prior results.

We begin this chapter with an overview of the RISE language and the setup of the case study. Then, we explain how we modeled the relevant operations of the RISE language in MimIR. We present two RISE programs along with the rewrite rules and the plugin configuration used to optimize them. Finally, we discuss the results of these optimizations and contrast them with results reported in the existing RISE literature.

4.1 Overview

At its core, RISE provides lambda-calculus constructs such as abstraction and application, as well as identifiers and literals. Its type system incorporates parametric polymorphism through type-level variables and a restricted form of dependent types in which types may contain expressions of a natural number type. Types in RISE that are particularly relevant to this case study and correspond directly to existing types in MimIR are dependent function types and array types. In addition, RISE contains a collection of high-level primitives for expressing data-parallel computations on multi-dimensional arrays, such as the map and reduce operations. (Hagedorn et al., 2020) RISE is not compiled in isolation but is typically coupled with the strategy language ELEVATE, which expresses rewrite-based optimization strategies for RISE programs. This leads to a separation of concerns in the compiler, in which the high-level semantics of the program are expressed in RISE, while its optimizations are specified in an ELEVATE strategy. (Steuwer et al., 2022)

To begin, we modeled the RISE operations relevant to our case study through axiomatic extensions in MimIR, which we present in detail in the following section. We evaluated our plugin on two concrete programs inspired by existing examples in the RISE literature. For each optimization, we defined a set of rewrite rules either as MimIR rules or as part of a dedicated ruleset. We present the rewrite rules and

the corresponding configuration of the eqsat plugin used to perform each optimization. Finally, we report whether the optimization targets were reached and present performance metrics, such as execution time and memory footprint.

The first example in our case study is inspired by the work of Schneider et al. (2025), in which the benefits of slotted e-graphs were demonstrated through RISE programs involving the map operation. A map operation can be split into multiple nested maps, while multiple nested maps can conversely be joined into a single map operation. These transformations are referred to as map fission and map fusion, respectively.

The second example in our case study is inspired by the work of Koehler, Trinder, and Steuwer (2022), in which the use of human-defined sketches to guide an equality saturation engine toward complex optimization targets was evaluated. The concrete RISE programs model a loop-tiling optimization for different numbers of dimensions, where the optimization target becomes increasingly large and difficult to discover as the number of dimensions grows.

4.2 Encoding RISE in MimIR

To express the RISE programs used in our case study, we modeled the relevant RISE primitives as MimIR axioms. A more complete implementation of RISE in MimIR would require a representation of the remaining primitives, together with the corresponding C++ normalizers and transformation phases. Since the purpose of our case study was to demonstrate the optimization of the selected RISE programs through the eqsat plugin, we chose to implement only the subset required by these examples. Listing 4.1 shows the resulting MimIR axioms.

```

1 plugin core;
2
3 axm %rise.o:      {A: *, B: *, C: *} → [B → C]      → [A → B]      → [A → C];
4 axm %rise.map:    {n: Nat}          → {S: *, T: *} → [S → T]      → [«n; S» → «n; T»];
5 axm %rise.transpose: {n: Nat, m: Nat} → {T: *}        → «n; «m; T»» → «m; «n; T»»;
6 axm %rise.join:    {n: Nat, m: Nat} → {T: *}        → «n; «m; T»» → «%core.nat.mul (n,
7 m); T»;
7 axm %rise.split:   {n: Nat}          → {T: *}        → [m: Nat]      → «%core.nat.mul (n,
m); T» → «n; «m; T»»;
```

LISTING 4.1: RISE primitives modeled in Mim

The `%rise.o` axiom encodes a function composition operator. It is polymorphic over the type-level variables A , B , and C , which are declared as implicit parameters through the curly brackets and inferred from the application context. It takes two functions of types $B \rightarrow C$ and $A \rightarrow B$ and composes them into a function of type $A \rightarrow C$. The `%rise.map` axiom encodes a map operator that receives a function of type $S \rightarrow T$ and produces a function that maps an array of length n with element type S to an array of the same length with element type T . The `%rise.transpose` axiom encodes a transpose operation that transforms a two-dimensional array of shape $n \times m$ into a two-dimensional array of shape $m \times n$. The `%rise.join` and `%rise.split` axioms encode mutually inverse operations. The `%rise.join` operation flattens a two-dimensional array of shape $n \times m$ into a one-dimensional array of length $n \cdot m$, while `%rise.split` converts such a flattened array into a two-dimensional array of shape $n \times m$.

4.3 Experimental Setup

We conducted the case study on a machine equipped with an Intel Core i7-7700K CPU with 4 cores and 8 threads at a base frequency of 4.2 GHz and 32 GB of RAM, running Windows 11 as its operating system. The implementation of the eqsat plugin utilized MimIR release version 0.1, egg version 0.11.0, and slotted-egraphs version 0.0.36.

For each example, we defined a configuration that specified an equality saturation backend, a cost function, and sources of rewrite rules. We did not limit the number of equality saturation iterations, e-classes, or e-nodes, specify a maximum timeout, or define a custom rewrite scheduler. The eqsat plugin then optimized the programs according to the configuration provided. We recorded the number of e-classes and e-nodes, the number of iterations, the execution time, the memory footprint, and whether the optimization goal was reached. We present the resulting measurements at the end of each case-study example.

4.4 Map Fusion and Fission

The first example in our case study is inspired by the work of Schneider et al. (2025), in which the benefits of slotted e-graphs were demonstrated by transforming RISE programs through map fusion and fission. The purpose of these rewrites was to prevent redundant computations and enable the reuse of intermediate results. In the following, we reproduce a similar transformation in MimIR using the slotted-egraphs backend of the eqsat plugin.

Optimization Target

The optimization for the first example is directly derived from Schneider et al. (2025) and is shown in (4.1).

$$\begin{aligned}
 & \text{map} \left(\text{map} \left(\lambda x. f_4 \left(f_3 \left(f_2 \left(f_1 x \right) \right) \right) \right) \right) y \\
 & \quad \longrightarrow^* \\
 & \text{map} \left(\text{map} \left(\lambda x. f_4 \left(f_3 x \right) \right) \right) \left(\text{map} \left(\text{map} \left(\lambda x. f_2 \left(f_1 x \right) \right) \right) y \right)
 \end{aligned} \tag{4.1}$$

Here, y represents a two-dimensional array in which two nested maps apply the functions f_1 , f_2 , f_3 , and f_4 to each element. The optimization target represents an equivalent program in which f_1 and f_2 are applied to the elements of y , producing an intermediate array. Then f_3 and f_4 are applied to the elements of this intermediate array. This transformation constitutes an optimization when the intermediate result produced by f_1 and f_2 can be reused elsewhere. Listing 4.2 shows how we modeled this transformation in Mim.

```

1  lam f1(x: Nat)@ff = %core.nat.add (x, 1);
2  lam f2(x: Nat)@ff = %core.nat.sub (x, 1);
3  lam f3(x: Nat)@ff = %core.nat.mul (x, 2);
4  lam f4(x: Nat)@ff = %core.nat.add (x, 2);
5
6  fun extern fission(y: «3; «4; Nat»») =
7    return ((%rise.map (%rise.map lam(x: Nat) = f4 (f3 (f2 (f1 x)))))) y);
8
9  fun extern fission_expected(y: «3; «4; Nat»») =
10   return ((%rise.map (%rise.map lam(x: Nat) = f4 (f3 x))) ((%rise.map (%rise.map lam(x:
    Nat) = f2 (f1 x))) y));

```

LISTING 4.2: Map-fission optimization in Mim

We defined the lambdas f_1 , f_2 , f_3 , and f_4 that correspond to the variables f_1 , f_2 , f_3 , and f_4 in (4.1). Each of these performs an arbitrary operation on its natural number input. We additionally set their partial-evaluation filter to `ff` to avoid normalization and ensure that the underlying IR retains the structure of the given example. The external function `fission` represents the initial program term, using the previously defined `%rise.map` axiom to represent the map operations. Its input parameter y is a two-dimensional array corresponding to the variable y in (4.1). The function `fission_expected` similarly represents the optimization target, in which the applications of f_1 and f_2 have been separated from those of f_3 and f_4 through map fission.

Rewrite Rules

The example utilized four rewrite rules, which we adopted from Schneider et al. (2025). The rules in (4.2) and (4.3) perform β -reduction and η -reduction, respectively. The map fusion rule in (4.4) transforms two nested maps applying the functions f and g into a single map applying a lambda that encodes their composition $f \circ g$. Conversely, the map-fission rule in (4.5) rewrites a map applying a lambda that encodes the composition $f \circ g$ into a lambda that applies f and g in separate nested maps.

$$(\lambda x. b) e \rightarrow b[x := e] \quad (4.2)$$

$$\lambda x. f x \rightarrow f \quad \text{if } x \notin FV(f) \quad (4.3)$$

$$\text{map } f (\text{map } g y) \rightarrow \text{map } (\lambda x. f (g x)) y \quad (4.4)$$

$$\text{map } (\lambda x. f (g x)) \rightarrow \lambda y. \text{map } f (\text{map } (\lambda x. g x) y) \quad \text{if } x \notin FV(f) \quad (4.5)$$

We implemented these rules as part of the ruleset `%eqsat.rise` in the slotted-egraphs backend. Listing 4.3 shows the implementation of the map fusion rule defined in (4.4). The remaining rules are similarly implemented in the `%eqsat.rise` ruleset.

```

1  fn map_fusion() -> Rewrite<Mim, MimAnalysis> {
2    let pat = "(app (app (app (app %rise.map ?n) (tuple (cons ?B (cons ?C nil)))) ?f) (app
    (app (app (app %rise.map ?n) (tuple (cons ?A (cons ?B nil)))) ?g) ?y))";
3    let output = "(app (app (app (app %rise.map ?n) (tuple (cons ?A (cons ?C nil)))) (lam $x
    (scope (lit tt) (app ?f (app ?g (var $x))))) ?y)";
4    Rewrite::new("map-fusion", pat, output)
5  }

```

LISTING 4.3: Map fusion rule in the %eqsat.rise ruleset

The `map_fusion()` function returns the corresponding rule using the ruleset interface described in [subsection 3.4.2](#). The left-hand side of the rule is modeled as the s-expression pattern stored in `pat`, which makes use of the previously defined `%rise.map` axiom to represent the map operation and expresses the variables f , g , and y as the pattern variables `?f`, `?g`, and `?y`, respectively. Notably, this pattern also encodes the implicit arguments of the `%rise.map` axiom through the pattern variable `?n` for the array length and `?A`, `?B`, and `?C` for the element types. The right-hand side is similarly represented and stored in `outpat`. It additionally introduces a lambda that binds the variable x , which is represented by the slot `$x`.

Configuration

The eqsat plugin was configured with the lambda shown in [Listing 4.4](#).

```
1 lam extern _config() =
2   (
3     %eqsat.slotted,
4     %eqsat.rulesets (%eqsat.rise),
5     %eqsat.reaches (fission, fission_expected),
6   );
```

LISTING 4.4: Configuration for map fusion and fission

We specified the slotted-egraphs backend with the `%eqsat.slotted` axiom and selected the `%eqsat.rise` ruleset with the application `%eqsat.rulesets (%eqsat.rise)`. Finally, we configured a reachability check by providing the start term `fission` and the goal term `fission_expected` through the application `%eqsat.reaches (fission, fission_expected)`.

Results

[Table 4.2](#) shows the result of optimizing the map-fission example with our equality saturation plugin.

Metric	Result
Target reached	Yes
Iterations	8
E-classes	2302
E-nodes	512
S-expression emission time	67 ms
Equality saturation time	4.22 s
MimIR reconstruction time	46 ms
Total execution time	4.61 s
Peak process memory	55.1 MiB

TABLE 4.1: Result of the map-fission optimization

The map-fission optimization target was reached after eight iterations of equality saturation, with a total execution time of approximately 4.6 seconds. The e-graph size, execution time, and memory measurements are listed in the rows beneath. The execution time measurements further distinguish between the time required to emit an s-expression representation of the MimIR program, perform equality saturation,

and reconstruct the optimized program in MimIR. Notably, the number of e-classes reported by the slotted-egraphs backend exceeds the number of e-nodes. This is inconsistent with the definitions of e-classes and e-nodes and appears to be caused by a bug in the computation of the e-class metric in the slotted-egraphs library.

4.5 Loop Tiling

The second example in our case study is inspired by the work of Koehler, Trinder, and Steuwer (2022), in which human-defined sketches are used to guide an equality saturation engine towards complex optimization targets. The paper presents a set of increasingly complex optimizations of RISE programs and compares the performance of an unguided equality saturation engine with that of a guided one. The optimization technique modeled in these examples is loop tiling. Loop tiling splits the iteration space of a loop into smaller units, called tiles, with the aim of improving data locality and cache utilization. In RISE, loops are expressed through map operations, allowing loop tiling to be achieved by dividing individual map operations into multiple nested maps.

Optimization Target

The first transformation, shown in (4.6), represents a one-dimensional loop-tiling optimization and is directly derived from Koehler, Trinder, and Steuwer (2022).

$$\begin{aligned} & \text{map } (n_1 \cdot 32) f \\ & \quad \longrightarrow^* \\ & \text{join} \circ (\text{map } n_1 (\text{map } 32 f)) \circ (\text{split } 32) \end{aligned} \tag{4.6}$$

Here, a single loop over the iteration space $n_1 \cdot 32$ is represented by a map applying the function f . The optimized program splits the one-dimensional input array of length $n_1 \cdot 32$ into a two-dimensional array of shape $n_1 \times 32$, corresponding to n_1 tiles of size 32, and applies f to each element through two nested maps. Finally, the two-dimensional array is flattened back into a one-dimensional array with a join operation. Note that the implicit array-length arguments of the map operations are explicitly provided here. In addition, function composition is represented by the infix operator $\circ$ for better readability. Listing 4.5 shows how we modeled this transformation in Mim.

```

1  lam f(x: Nat) = %core.nat.add (x, 1);
2
3  fun extern tile(n1: Nat) =
4    let arg = <%core.nat.mul (n1, 32); 2>;
5    let mapper = %rise.map @(<%core.nat.mul (n1, 32)>) f;
6    return (mapper arg);
7
8  fun extern tile_expected(n1: Nat) =
9    let arg = <%core.nat.mul (n1, 32); 2>;
10   let mapper = %rise.o (%rise.join @(<n1, 32>) @Nat) (%rise.o (%rise.map @n1 (%rise.map @32
11     f)) (%rise.split @n1 @Nat 32));
    return (mapper arg);

```

LISTING 4.5: One-dimensional loop-tiling optimization in Mim

We defined the lambda f that corresponds to the variable f and performs an arbitrary operation on its input. As in the previous case-study example, we define the initial program in the `tile` function, whose input parameter `n1` corresponds to n_1 . The variable `arg` is instantiated with a pack of length $n_1 \cdot 32$ and serves as a concrete input to the `map` operation. This input is not shown in (4.6). We also define the mapper variable, whose definition corresponds to the initial program. It applies the `%rise.map` axiom to the implicit array-length argument `%core.nat.mul (n1, 32)` and the function f . In Mim, explicitly provided implicit arguments are denoted by an `@` prefix. The `tile_expected` function represents the optimization target through its mapper variable. Here, we additionally needed to specify implicit arguments for the `%rise.join` and `%rise.split` axioms that could not be inferred from the context.

Since the one-dimensional loop-tiling optimization requires only a single rewrite rule, we also derived a more complex two-dimensional loop-tiling example from Koehler, Trinder, and Steuwer (2022), which is shown in (4.7).

$$\begin{aligned}
 & \text{map } (n_1 \cdot 32) (\text{map } (n_2 \cdot 32) f) \\
 & \quad \longrightarrow^* \\
 & \text{join} \circ (\text{map } n_1 (\text{map } 32 \text{ join})) \circ (\text{map } n_1 \text{ transpose}) \circ \\
 & \quad (\text{map } n_1 (\text{map } n_2 (\text{map } 32 (\text{map } 32 f)))) \circ \\
 & \quad (\text{map } n_1 \text{ transpose}) \circ (\text{map } n_1 (\text{map } 32 (\text{split } 32))) \circ (\text{split } 32)
 \end{aligned} \tag{4.7}$$

We implemented this two-dimensional loop-tiling example analogously to the preceding one. Since its implementation is considerably larger, while following the same structure, we omit it for brevity.

Rewrite Rules

The example utilized five rewrite rules, which we adopted from Koehler, Trinder, and Steuwer (2022). The rules in (4.8) and (4.9) encode bidirectional associativity for the function composition operator $\circ$. Map fission is expressed by the rule in (4.10). The rule in (4.11) swaps the application order of two nested maps by inserting transpose operations before and after them. Finally, the rule in (4.12) splits a single map operation into two nested maps by surrounding it with `split` and `join` operations, as in the one-dimensional loop-tiling example.

$$a \circ (b \circ c) \rightarrow (a \circ b) \circ c \tag{4.8}$$

$$(a \circ b) \circ c \rightarrow a \circ (b \circ c) \tag{4.9}$$

$$\text{map } (a \circ b) \rightarrow \text{map } a \circ \text{map } b \tag{4.10}$$

$$\text{map } n_1 (\text{map } n_2 f) \rightarrow \text{transpose} \circ \text{map } n_2 (\text{map } n_1 f) \circ \text{transpose} \tag{4.11}$$

$$\text{map } (n_1 \cdot n_2) f \rightarrow \text{join} \circ \text{map } n_1 (\text{map } n_2 f) \circ \text{split} \tag{4.12}$$

We implemented these rewrite rules as the Mim rules shown in Listing 4.6.

```

1 rule compose_assoc_1(A B C D: *, a: C → D, b: B → C, c: A → B):
2   %rise.o a (%rise.o b c)
3 => %rise.o (%rise.o a b) c;
4
5 rule compose_assoc_2(A B C D: *, a: C → D, b: B → C, c: A → B):
6   %rise.o (%rise.o a b) c
7 => %rise.o a (%rise.o b c);
8
9 rule map_fission(A B C: *, a: B → C, b: A → B, n: Nat):
10  %rise.map @n (%rise.o a b)
11 => %rise.o (%rise.map @n a) (%rise.map @n b);
12
13 rule transpose_maps(S T: *, f: S → T, n1 n2: Nat):
14  %rise.map @n1 (%rise.map @n2 f)
15 => %rise.o (%rise.transpose @n2, n1) @T (%rise.o (%rise.map (%rise.map f))
16   (%rise.transpose @n1, n2) @S));
17
18 rule split_map(S T: *, f: S → T, n1 n2: Nat):
19  %rise.map @(%core.nat.mul (n1, n2)) f
20 => %rise.o (%rise.join @n1, n2) @T (%rise.o (%rise.map (%rise.map f)) (%rise.split @n1
21   @S n2));

```

LISTING 4.6: Loop-tiling rewrite rules in Mim

Here, the rules are defined in the same order as presented above. For example, the first $\circ$ -associativity rule in (4.8) is implemented by the rule `compose_assoc_1`. This rule receives the type-level parameters A , B , C , and D , as well as the functions a , b , and c over these types. All of these parameters are internally modeled as pattern variables, as explained in subsection 3.4.3. The left-hand side of the rule $a \circ (b \circ c)$ is represented by the expression `%rise.o a (%rise.o b c)`, while its right-hand side $(a \circ b) \circ c$ is represented by the expression `%rise.o (%rise.o a b) c`.

Configuration

Listing 4.7 shows the configuration of the `eqsat` plugin that we defined for the loop-tiling optimization.

```

1 lam extern _config() =
2   (
3     %eqsat.slotted,
4     %eqsat.rules_kind
5       (compose_assoc_1,
6        compose_assoc_2,
7        map_fission,
8        transpose_maps,
9        split_map),
10    %eqsat.reaches (tile, tile_expected),
11  );

```

LISTING 4.7: Configuration for loop tiling

As in the previous example, we utilized the slotted-egraphs backend and specified a reachability check between the initial term `tile` and the goal term `tile_expected`. In this case, however, we specified the previously defined Mim rules by applying the `%eqsat.rules_kind` axiom to a tuple containing the relevant rules.

Results

Table 4.2 shows the results of optimizing the one- and two-dimensional loop-tiling examples with our equality saturation plugin.

Metric	Tile1D	Tile2D
Target reached	Yes	Yes
Iterations	3	8
E-classes	154	618617
E-nodes	111	146895
S-expression emission time	0.86 s	3.84 s
Equality saturation time	0.31 s	36.77 min
MimIR reconstruction time	0.09 s	0.38 s
Total execution time	1.53 s	36.97 min
Peak process memory	41.6 MiB	6138.1 MiB

TABLE 4.2: Results of the loop-tiling optimizations

The one-dimensional loop-tiling optimization target was reached after three iterations of equality saturation, with a total execution time of about 1.5 seconds. The two-dimensional optimization target was reached after eight iterations, with a total execution time of approximately 37 minutes. As in the map-fission evaluation, the e-class counts are affected by the previously described bug in the slotted-egraphs library.

4.6 Discussion

Both the map-fission and loop-tiling examples successfully reached the defined optimization targets with our equality saturation plugin. Each example utilized the RISE primitives that we encoded as MimIR axioms, demonstrating the ease with which MimIR can be extended with the abstractions of a concrete language. The examples also made use of both mechanisms for defining rewrite rules: the map-fission rules were defined as part of the `%eqsat.rise` ruleset, while the loop-tiling rules were expressed directly in MimIR. In both cases, the rules were passed to the plugin through its configuration, which additionally allowed the concrete equality saturation backend and other parameters to be selected. The case study demonstrates that optimizations based on examples from the existing RISE literature can be expressed in MimIR and performed with the eqsat plugin. This suggests that the eqsat plugin can also be used to optimize the terms of other languages modeled in MimIR.

While all optimization targets were reached, there were substantial differences in the measured performance between the individual examples. The less complex transformations of map fission and one-dimensional loop tiling were discovered within a few seconds and resulted in relatively small e-graphs, as indicated by their e-node counts. In contrast, the two-dimensional loop-tiling optimization required approximately 37 minutes and produced a substantially larger e-graph. The number of rewrite steps required to produce the optimized term from the initial term helps explain this difference. While the one-dimensional loop tiling required only a single rewrite step, the two-dimensional example required 16 distinct rewrite steps. Introducing an intermediate goal between the initial and goal terms of the two-dimensional example significantly reduced the search time: the transformation from

the start term to the intermediate term was discovered in approximately 37 seconds, while the transformation from the intermediate term to the goal term was discovered in around 25 seconds. This is also the motivation behind guided equality saturation, where intermediate guides divide a complex optimization into a sequence of smaller searches.

We further measured the isolated runtimes for the emission of an s-expression representation of the MimIR program, equality saturation, and the reconstruction of the optimized terms in MimIR. The purpose of these measurements was to gauge the runtime overhead incurred by the additional abstraction layer between MimIR and an equality saturation library. While the reconstruction times remained well under a second for every example in the case study, s-expression emission took noticeably longer for larger programs. Emitting the s-expression representation of the two-dimensional loop-tiling example took approximately four seconds. We attribute this comparatively high emission time to two aspects of its implementation. First, already emitted s-expressions are not cached for subsequent reuse. Second, and more significantly, expensive string transformations are repeatedly performed to produce an indented s-expression. These overheads could be mitigated by emitting an unformatted s-expression, since its primary purpose is to serve as an intermediate representation which does not need to be human-readable. This would similarly enable the use of caching, which we previously omitted because cached s-expressions retain their original indentation and may therefore be incorrectly indented when reused at a different nesting level. For the reconstruction stage, the measured performance appears adequate, although it could potentially be improved by consolidating the two reconstruction traversals. Overall, the measurements indicate that the overhead incurred by the additional abstraction layer is relatively small and could be further reduced by optimizing the s-expression emitter.

A comparison of the map-fission results in our case study with those reported by Schneider et al. (2025) reveals several differences. The e-graph produced in our example contained 512 e-nodes, compared with the 214 e-nodes reported in their work. We attribute this difference in part to the larger s-expressions required to represent MimIR programs. Equality saturation also required approximately four seconds, compared with 220 milliseconds in Schneider et al. (2025). However, these measurements are not directly comparable, as our measurement includes the collection of rulesets, the conversion of rule s-expressions, the parsing of large, type-annotated s-expressions, and an initial transfer of types to the e-class analysis data of their corresponding terms. These additional steps may explain a large part of the difference in runtime. Finally, our measurements show a higher peak process memory usage of 55 MiB, compared with 6 MB in their work. Since our measurement captures the peak memory consumption of the complete process, including s-expression emission and reconstruction, a higher memory usage is to be expected.

Comparing our loop-tiling measurements with those reported by Koehler, Trinder, and Steuwer (2022) reveals a substantial difference in memory consumption for the two-dimensional case. While our optimization target was reached after eight iterations with a peak memory consumption of approximately 6.1 GiB, Koehler, Trinder, and Steuwer (2022) report seven iterations and a memory footprint of only 1 MB. The number of iterations is comparable, while the measured memory consumption differs by several orders of magnitude. We are unsure of the exact cause of this difference.

In this case study, we evaluated the effectiveness of our equality saturation plugin on selected examples from the RISE literature. The evaluation aimed to assess the plugin on a concrete workload rather than comparing the performance of the

egg and slotted-egraphs backends. We conclude that, for the evaluated examples, the eqsat plugin was capable of reproducing the desired transformations, while the additional abstraction layer between MimIR and the equality saturation backends introduced a relatively small performance overhead. Overall, the case study shows that equality saturation can be integrated into MimIR and used for rewrite-based optimizations of languages modeled in MimIR.

Chapter 5

Conclusion

As **DSLs** are increasingly relied on to generate highly optimized code for specific hardware and domains, many of these languages, particularly those based on the lambda-calculus, could benefit from integrating equality saturation into their optimization procedures. Equality saturation was initially proposed to address the phase-ordering problem, which arises because the order in which transformations are applied directly influences the quality of the generated code. Determining an optimal order, however, is not trivial and often requires exploring a substantial number of rewrite sequences. The use of e-graphs and equational rewriting in equality saturation made the exploration of large transformation spaces feasible. While the MimIR compiler framework already provides the capabilities for modeling a large variety of **DSLs** through domain-specific **IR** extensions, equality saturation previously had to be integrated separately for each language that required it. We aimed to address this challenge in this work by extending MimIR with a language-agnostic equality saturation plugin. The resulting eqsat plugin facilitates rewrite-based optimization of **DSLs** expressed in MimIR by integrating two equality saturation backends based on egg and slotted-egraphs.

To enable the interaction between MimIR and the egg and slotted-egraphs equality saturation libraries, we integrated an s-expression emitter into the MimIR compiler. The s-expression representation served as an abstraction layer that bridged the gap between MimIR terms and the generic equality saturation mechanisms offered by the libraries. To this end, we encoded the lambda nesting structure of MimIR programs, as derived from the nesting tree, and equipped the s-expressions with type annotations that allowed type information from MimIR to be transferred to e-class analysis data.

We then defined a set of **IR** extensions through the eqsat plugin that facilitate the configuration and orchestration of equality saturation directly from MimIR. This included options for selecting the backend and cost functions, specifying sources of rewrite rules, and defining reachability checks. These configuration axioms are bundled in a MimIR-defined lambda in which a developer specifies the parameters for a particular equality saturation workflow.

To facilitate the definition of rewrite rules, we implemented two mechanisms for their construction. Firstly, rewrite rules can be defined as a group of related rules, which we call a ruleset and represent with corresponding axioms in MimIR. Secondly, rewrite rules can be specified directly through the rule **IR** construct in MimIR. While the implementation of a ruleset requires greater effort, it enables developers to make use of the full expressiveness of the egg and slotted-egraphs rewrite **APIs**, including conditional and dynamic rules in addition to purely syntactic rules. Rewrite rules defined in MimIR are limited in the sense that they are purely syntactic, but offer the benefit of being directly integrated into the **IR**. This allows them

to be dynamically constructed at runtime, as opposed to rulesets, which require re-compilation with every change.

To model MimIR as a concrete term language for egg and slotted-egraphs, we defined corresponding language variants that faithfully represent its IR constructs in accordance with the s-expression representation of MimIR. We used the abstractions provided by the libraries to populate an e-graph with a MimIR input program and attempt to saturate it with the rewrite rules sourced from the selected rulesets and provided MimIR rules. A lowest-cost term is then extracted for every MimIR root term according to the configured cost function. We also implemented additional functionality in these backends, such as equality-saturation-driven reachability checks and an e-class analysis for maintaining the type information associated with MimIR terms.

Finally, we implemented a rewrite phase bundled with the eqsat plugin that is responsible for the high-level orchestration of equality saturation and the reconstruction of the optimized MimIR program through two dedicated traversals. During reconstruction, an initialization traversal first creates the required binders before their references, while a subsequent conversion traversal reconstructs the remaining MimIR definitions from the recursive expressions representing the optimized program.

We evaluated our equality saturation plugin on two concrete RISE program transformations derived from existing literature. These consisted of a map-fission optimization and a loop-tiling optimization. We used MimIR’s capabilities for IR extensions to model the relevant subset of RISE primitives in MimIR. The rewrite rules involved in the transformations were expressed through a ruleset in the map-fission example and MimIR-defined rules in the loop-tiling example. We found that every expected optimization target was discovered through our plugin. Whereas the less complex map-fission and one-dimensional loop-tiling optimizations were discovered within a few seconds, the more complex two-dimensional loop-tiling optimization required a much larger search and took approximately 37 minutes. We further measured the overhead introduced by the abstraction layer between MimIR and the equality saturation backends, including s-expression emission and the reconstruction of terms in MimIR. We found that this overhead was comparatively small, with reconstruction times remaining well below one second in all examples and the un-optimized s-expression emitter requiring no more than approximately four seconds.

Throughout this work, we identified several limitations of our implementation. The first concerns the e-graph representation of variables in the egg backend, for which we relied on MimIR’s globally unique naming mechanism. A shortcoming of this approach is that α -equivalent terms do not share the same representation, which could have been achieved with a representation based on de Bruijn indices instead. Furthermore, this approach required the implementation of a free-variable analysis for binder-sensitive rewrite rules and the dynamic generation of unique variable names for rewrite rules introducing new binders. To circumvent these shortcomings, we chose to supplement the eqsat plugin with a further backend implemented in slotted-egraphs, in which these challenges are addressed by the library’s handling of variables and binders in a slotted e-graph. A potential improvement to the egg backend would be to replace its name-based variable representation with one based on de Bruijn indices and implement the necessary index-shifting operations. A substantial benefit of this approach is the canonical representation of α -equivalent terms, which enables greater sharing in an e-graph. However, implementing the associated index-shifting operations would still require a non-trivial implementation effort.

Another limitation we encountered in our e-class analysis for maintaining type information is that context-dependent types could not be maintained correctly during equality saturation. In the **CoC**, types can depend on variables that are defined in a surrounding context. If a type is stored in isolation as e-class analysis data, changes to the binder structure of the surrounding context cannot be accounted for in the type. During this work, we were unable to identify an elegant solution to this problem. We consider this limitation the most significant of those described, as its practical implications are that the eqsat plugin cannot reliably maintain type information for certain program transformations involving context-dependent types. While such programs were not encountered in the concrete examples of our work, addressing this limitation remains an important direction for future work.

We encountered two further implementation limitations related to the s-expression emitter and MimIR reconstruction. The first is the relatively poor performance of the s-expression emitter, which repeatedly performs expensive string transformations to produce human-readable, indented s-expressions and does not cache previously emitted s-expressions. The second concerns the reconstruction of terms in MimIR through two isolated traversals per recursive expression, which could be consolidated into a single traversal. However, as our evaluation shows, the runtime overhead of the existing approach is already comparatively small. The first of these performance-related shortcomings could be addressed by modifying the s-expression emitter to emit an unindented representation, thereby eliminating the need for expensive string operations, while the second could be addressed through a consolidation of the two traversals into one.

Lastly, while we were able to reproduce two optimizations of RISE programs from the existing literature, we did not evaluate our plugin on a larger variety of **DSLs** modeled in MimIR, and both optimizations were performed using the slotted-egraphs backend. Therefore, our evaluation only establishes the plugin's ability to reproduce the selected transformations and the performance measurements only apply to the specific examples using slotted-egraphs. To evaluate the applicability and performance of the eqsat plugin more broadly, an evaluation involving a variety of **DSLs** and comparisons between the egg and slotted-egraphs backends is required.

Overall, we believe that we have shown in this work that a language-agnostic equality saturation mechanism can be integrated into MimIR through a dedicated plugin and that the resulting plugin is capable of reproducing non-trivial optimizations for **DSLs** modeled in MimIR.

Bibliography

- Baader, Franz and Tobias Nipkow (1998). *Term Rewriting and All That*. Cambridge University Press.
- Barendregt, Henk P (1991). “Introduction to generalized type systems”. In.
- Barthou, Denis and Sid Touati (May 2006). “On the Decidability of Phase Ordering Problem in Optimizing Compilation”. In: *Proceedings of the 3rd conference on Computing frontiers (CF’06)*. Ischia, Italy: ACM, pp. 147–156. DOI: [10.1145/1128022.1128042](https://doi.org/10.1145/1128022.1128042). URL: <https://hal.science/hal-00141063>.
- Coquand, Thierry and Gérard Huet (1988). “The calculus of constructions”. In: *Information and Computation* 76.2, pp. 95–120. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3). URL: <https://www.sciencedirect.com/science/article/pii/0890540188900053>.
- Cousot, Patrick and Radhia Cousot (1977). “Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints”. In: *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. POPL ’77. Los Angeles, California: Association for Computing Machinery, 238–252. ISBN: 9781450373500. DOI: [10.1145/512950.512973](https://doi.org/10.1145/512950.512973). URL: <https://doi.org/10.1145/512950.512973>.
- de Bruijn, N.G (1972). “Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem”. In: *Indagationes Mathematicae (Proceedings)* 75.5, pp. 381–392. ISSN: 1385-7258. DOI: [https://doi.org/10.1016/1385-7258\(72\)90034-0](https://doi.org/10.1016/1385-7258(72)90034-0). URL: <https://www.sciencedirect.com/science/article/pii/1385725872900340>.
- Esmailzadeh, H. et al. (2011). “Dark silicon and the end of multicore scaling”. In: *International Symposium on Computer Architecture*, pp. 365–376. URL: <https://www.microsoft.com/en-us/research/publication/dark-silicon-and-the-end-of-multicore-scaling/>.
- Hagedorn, Bastian et al. (2020). “Achieving high-performance the functional way: a functional pearl on expressing high-performance optimizations as rewrite strategies”. In: *Proceedings of the ACM on Programming Languages* 4.ICFP, pp. 1–29.
- Koehler, Thomas, Phil Trinder, and Michel Steuwer (2022). *Sketch-Guided Equality Saturation*.
- Lattner, Chris et al. (2021). “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation”. In: *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14. DOI: [10.1109/CGO51591.2021.9370308](https://doi.org/10.1109/CGO51591.2021.9370308).
- Leia, Roland and Johannes Max Griebl (June 2026). “SSA without Dominance for Higher-Order Programs”. In: *Proc. ACM Program. Lang.* 10.PLDI. DOI: [10.1145/3808286](https://doi.org/10.1145/3808286). URL: <https://doi.org/10.1145/3808286>.
- Leia, Roland et al. (Jan. 2025). “MimIR: An Extensible and Type-Safe Intermediate Representation for the DSL Age”. In: *Proc. ACM Program. Lang.* 9.POPL. DOI: [10.1145/3704840](https://doi.org/10.1145/3704840). URL: <https://doi.org/10.1145/3704840>.

- Lücke, Martin, Michel Steuwer, and Aaron Smith (2021). “Integrating a functional pattern-based IR into MLIR”. In: *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*. CC 2021. Virtual, Republic of Korea: Association for Computing Machinery, 12–22. ISBN: 9781450383257. DOI: [10.1145/3446804.3446844](https://doi.org/10.1145/3446804.3446844). URL: <https://doi.org/10.1145/3446804.3446844>.
- McCarthy, John (1960). “Recursive functions of symbolic expressions and their computation by machine, part I”. In: *Communications of the ACM* 3.4, pp. 184–195.
- Nelson, Greg and Derek C Oppen (1980). “Fast decision procedures based on congruence closure”. In: *Journal of the ACM (JACM)* 27.2, pp. 356–364.
- Pierce, Benjamin C., ed. (Dec. 2004). *Advanced Topics in Types and Programming Languages*. The MIT Press. ISBN: 9780262281591. DOI: [10.7551/mitpress/1104.001.0001](https://doi.org/10.7551/mitpress/1104.001.0001). URL: <https://doi.org/10.7551/mitpress/1104.001.0001>.
- Pierce, Benjamin C et al. (2002). *Types and programming languages*. Vol. 1. 19.3. MIT press Cambridge.
- Schneider, Rudi et al. (2025). “Slotted E-Graphs: First-Class Support for (Bound) Variables in E-Graphs”. In: *Proceedings of the ACM on Programming Languages* 9.PLDI, pp. 1888–1910.
- Steuwer, Michel et al. (2022). *RISE & Shine: Language-Oriented Compiler Design*. arXiv: [2201.03611](https://arxiv.org/abs/2201.03611) [cs.PL]. URL: <https://arxiv.org/abs/2201.03611>.
- Tate, Ross et al. (2011). “Equality Saturation: A New Approach to Optimization”. In: *Logical Methods in Computer Science* Volume 7, Issue 1, 10. ISSN: 1860-5974. DOI: [10.2168/LMCS-7\(1:10\)2011](https://lmcs.episciences.org/1016). URL: <https://lmcs.episciences.org/1016>.
- Willsey, Max et al. (Jan. 2021). “egg: Fast and extensible equality saturation”. In: *Proc. ACM Program. Lang.* 5.POPL. DOI: [10.1145/3434304](https://doi.org/10.1145/3434304). URL: <https://doi.org/10.1145/3434304>.